LINEAR Data Structure:

Calpephore Nkikabahizi

UNIT-III DATA STRUCTURE

INTRODUCTION

Data structure **data + structure**

- **Data** is collection /raw of unorganized facts, statistics, figures ,or bits.
- Data is unprocessed and meaningless information
- Organized data resulted in information, then information is the result of analyzing and interpreting pieces of data.
- Data may be organized in **many different ways logical** or mathematical model of a program.
- Organization of data means classification, tabulation, graphical presentation and diagrammatic presentation of data.
- The methods that we use to organize data include **classification, tabulation, graphical and diagrammatic presentation**.

Cont’

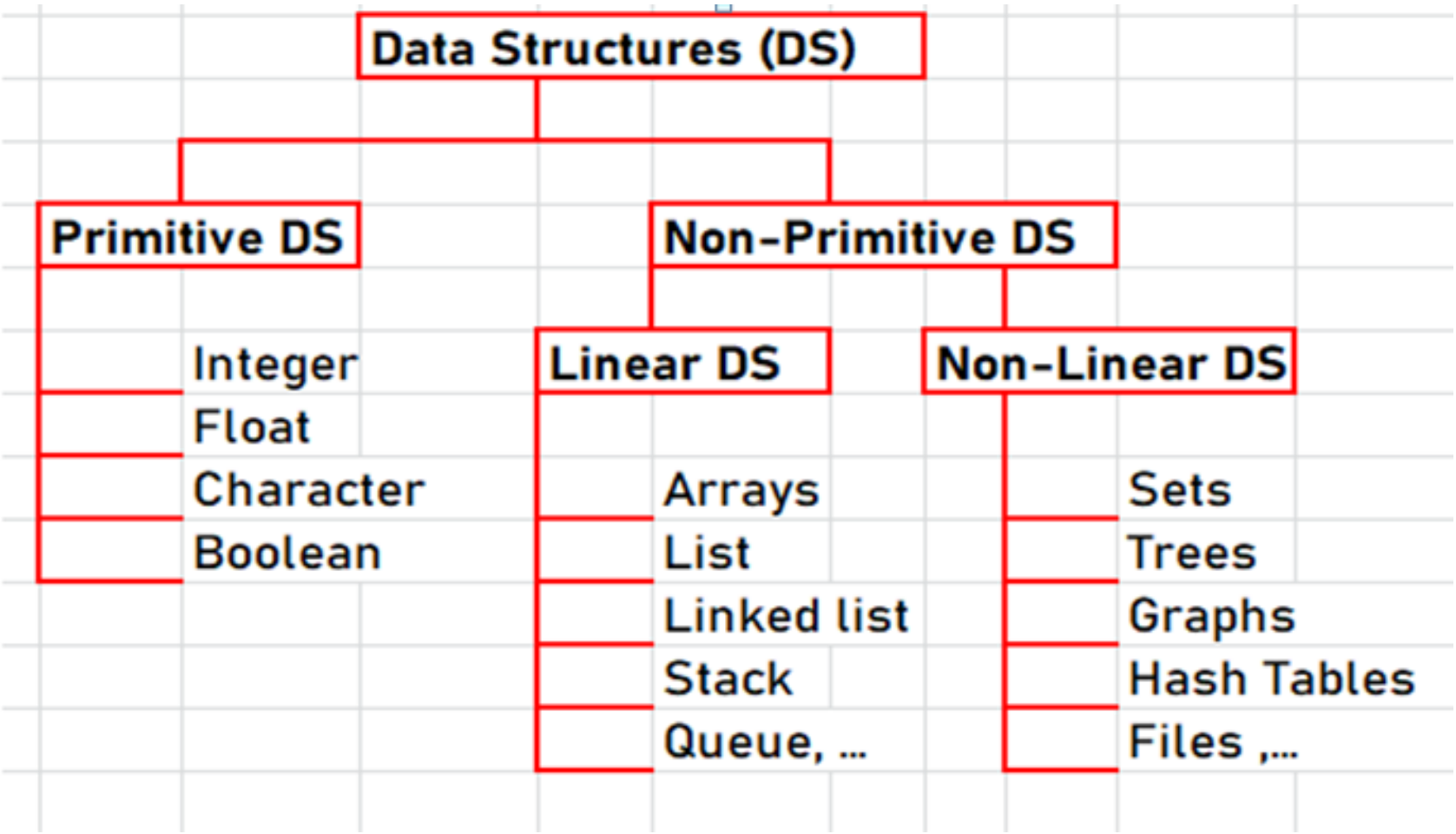
- A **data structure** is **a** specialized format **for organizing, processing, retrieving and storing data**. It is a way of arranging data on a computer so that it can be accessed and updated efficiently
- **Data Structure=Organized data +Allowed operations**
- Data Structure involves two complementary goals.:

1. To identify and develop useful, mathematical entities and operations and to determine what class of problems can be solved by using these entities and operations.

2. To determine representation for those abstract entities to implement abstract operations on this concrete representation.

Data structure classification

The DS has the classes tabulated below:



Primitive Data VS Non-primitive **structures**

- **Primitive Data structures** are directly supported by the language ie; any operation is directly performed in these data items.
- Ex: integer, Character, float, real numbers etc.
- Non-primitive data types are not defined by the programming language, but **are instead created by the programmer**.

Primitive DS

- Primitive data structures are basic simplest data types that are **used in programming**
- Primitive data structures have several key features that make them useful in programming. These attributes are: **Size:** They have fixed size. **speed:** they can be processed quickly by the computer; they us a minimal amount of memory (**memory efficiency**). They are also the same across different programming languages and platforms, which makes code more portable.
- Portability

Primitive DS Syntax

➢int

The int data type is a primitive DS that is used to represent integer values. It can store both positive and negative whole numbers. The **int** data type has a fixed size of **4 bytes in memory**.

Syntax in	Example
C++	int myInt = 25;
Java	int myInt = 25;
Python	myInt = 25;

Primitive DS Syntax

➢ Float

- The float data type is a primitive DS that is used to represent floating-point numbers. It is used to store real numbers that require a decimal point, such as 3.14 or -2.5. The float data type has a fixed size of 4 bytes in memory.

Syntax in	Example
C++	float myFloat = 3.14;
Java	float myFloat = 5.20f;
Python	myFloat = 7.14, float(50.76)

Primitive DS Syntax

➢ Double

- The double data type is similar to the float data type but can **store larger decimal values with more precision**. It is a primitive data structure that is used to represent floating-point numbers with double precision. The double data type has a fixed size **of 8 bytes in memory.**

Syntax in	Example
C++	double myDouble = 3.14159;
Java	double myDouble = 3.14159d;
Python	**does not have a specific "double" data type** like some others

Primitive DS Syntax

➢ Long

- The long data type is a primitive data structure that is used to represent integer values that **require more memory than an int**. It can store larger whole numbers, such as 4147583647 or -6147480648. The long data type has a fixed size of 8 bytes in memory.

Syntax in	Example
C++	long myLong = 4147483647;
Java	long myLong = 2147483647;
Python	By default, all the integers are long

Primitive DS Syntax

➢ Short

- The short data type is a primitive DS that is used to represent integer values that **require less memory than an int.** It can store smaller whole numbers, such as 42767 or -42768. The short data type **takes only 2 bytes in memory.**

Syntax in	Example
C++	Short int num = 42767;
Java	short num = 32767;
Python	

Primitive DS Syntax

- **Char**
- The char data type is a primitive data structure that represents a single character, such as a **letter, digit, or symbol**. The char data type has a fixed size of two bytes.

Syntax in	Example
C++	char myChar = 'A'; //**string** s1=‘hi’
Java	char myChar = 'A'; String s="javat";
Python	str

Primitive DS Syntax

➢ Bool

- The bool data type is a primitive data structure that represents a **logical value**, which can be either true or false. In most programming languages, the bool data type has a fixed size of one byte.

Syntax in	Example
C++	bool a = true
Java	bool a = false
Python	a = True

Primitive DS Syntax

➢ Byte

- The byte data type is a primitive DS that represents an 8-bit signed integer. In Java, the byte data type is represented using the keyword "byte" and has a size of 1 byte. It is useful for storing small integers in memory and is often used in situations where memory usage is critical.

Syntax in	Example
C++	
Java	byte myByteVariable = 127
Python	

- In this example, "myByteVariable" is a byte variable that has been assigned the value 127. Note that the range of values that can be stored in a byte is -128 to 127, inclusive.

Linear DS VS non-linear DS

Linear data structure

- A data structure is said **to be linear if its elements form a sequence**. Element are sequentially organize
- A linear list is a list that displays the **relationship of adjacency between elements**.
- They are easy to implement in the computer's memory.

Nonlinear data structures

- In nonlinear data structures, a data element can be **attached to several other data elements to represent specific relationships that exist among them**.
- Selecting one data structure type over the other should be done carefully by considering the relationship among the data elements that needs to be stored.
- It is difficult to implement it in computer's memory

Classification according to collections

- Collection types represent different ways to collect data, such as hash tables, queues, stacks, bags, dictionaries, and lists.
- These collections may be classified according to the number of values.
- We defined two classes which are I-list and I-dictionary.
- I-list has **one value**, while **I-dictionary has two values.**

Cont’

Ilist

This collection has unique value

Example :

- Array
- List
- Queue
- Stack
- sets

Idictionary

- The collection types, each has **key and value**
- **Example:**

  Dictionary

  Hash table

  Sorted list

Operations on DS

- Following operations can be performed on the data structures:

1. Create
2. Traversing
3. Inserting
4. Deleting
5. Sorting
6. Searching (Object, index)
7. Update
8. Display
9. math operators, +,-,*,/

UNIT II-1: ARRAY

- **I.1.Introduction :**
- In this chapter we discuss: array, list ,tuple,
- An ***array*** is a container object that holds a **fixed number of values of a single type**
- An array is a **random access data structure**, where **each element can** be **accessed directly** and in constant time
- The length of an array is established when the array is created.
- Each item in an array is called an *element*, and each element is accessed by its **numerical *index*, single identifier name**.

Array Representation

- Arrays are represented as **a collection of buckets where each bucket stores one element**.
- The elements are in contiguous format
- These buckets are indexed from '0' to 'n-1', where n is the size of that particular array.
- For example1, an array with size 9 will have buckets indexed from 0 to 8. or 1 to 9.

Indices →	1	2	3	4	5	6	7	8	9
size:10	2	4	6	7	10	15	11	4	8

Array Representation

P. Language	Example of arrays
Java	1. String[] courses = {“DS", “Math", “DM", “English"}; System.out.println(Courses[0]); 2. int[] myNum = {10, 20, 30, 40};
C++	string courses[4] = {“DS", “Math", “DM", “English"}; int myNum[5] = {10, 20, 30, 40,50};
Python	courses = [“DS", “Math", “DM", “English“]; myNum = [10, 20, 30, 40,50];

Cont’

- Each element is accessed by using its index

Syntax in	
C++	int arrSize = sizeof(arr)/sizeof(arr[0]); Or <<end(arr)-begin(arr)>>
Java	A.length
Python	len (A)

- Example2:
- A typical illustration of random access is a book –
- Each page of the book **can be open independently of others**.
- Random access is critical to many algorithms, for example**, binary search**.

Array data type

- The array can be classified according to primitive structure of data:
- Empty array : E=[]
- Integer array (int): L=[2, 4, 6, 8, 10]
- Double array (double/float): L=[1.0, 2.0, 3.0, 4.0, 5.0]
- String array (str): S=['one', 'Two', 'Three']
- Character array C=['o', 'n', 'o']
- Boolean array (bool) B=[True, False, False, True]
- Byte array : to covert data to string

Types of arrays

- There are three different kinds of arrays: **indexed arrays (** simple list**), multidimensional arrays (**matrix**), and associative arrays (**dictionary**).**

1. Creating indexed array:

- To create an array, we create an empty array whereby you push (java)/or append (python)
- Example: in array,

L=[“one”, “Two”, “three”, “Four”]

1.1.1 Simple array:

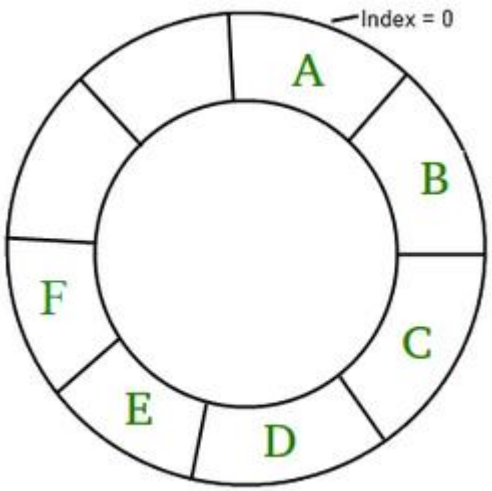
- Writing an simple array

Programming Language	Syntax +Example
C++	int arr[4]={3,8,9,1}; char arr[6]={'c', 'd', 'e‘, ‘k', ‘l', ‘n‘, }; float arr[7]= { 10.5, 7.5, 3.0, 6.8, 7.00, 4.5, 5.7 }
Java	/ The syntax of declaring a static array is: <data type><variable name>[] = {<data1>, <data2>,.....<dataN> }; int arr[] = { 2, 0, 6, 9, 7, 3 };

Python	arr = [10, 2,9 , 30] # This array will store integer
	arr2 = ['c', 'd', 'e‘, ‘w'] # This array will store characters
	arr3 = [28.5, 36.0, 40.2] # This array will store floating elements

Circular array list :

- An array is called circular if we consider the **first** element as next of **the last element**.
- **An example problem :**
  Suppose n people are sitting at
- a circular table with names
- A, B, C, D, … Given a name,
- we need to print all n people
- (in order) starting from the given name.



Circular representation of an array

Cont’

- For example, consider 6 people A B C D E F and given name as ‘D’. People sitting in a circular manner starting from D are D E F A B C.
  A **simple solution** is to create an auxiliary array of size 2*n and store it in another array [**Extra space]**
- If D is at starting position, count six so you don’t exceed the size ;If C is at a starting position; do the same

A	B	C	D	E	F	A	B	C	D	E	F
			D	E	F	A	B	C			

1.1.2 Multidimensional Arrays

2. Creating Multidimensional Arrays

- In programming, you can implement arrays as nested arrays that are essentially arrays of arrays. **Nested arrays**, also known as **multidimensional arrays**, are very similar to matrices or grids.
- **Applications:**
- Example1: A chess board, for example, is a grid of eight columns and rows; you could use the chess board as a model for an array that contains eight elements, each of which is also an array that contains eight elements.

1.1.2 Multidimensional Arrays

- **Example2:**

A **list of tasks** that is stored as an indexed array of strings.

Array = ["wash dishes", "take out trash"];

If you want to store a separate list of tasks for each day of the week, you **can create a multidimensional array** with one element for each day of the week. Each element contains an indexed array that stores the list of tasks.

1.1.2 Multidimensional Arrays

- **Example3:**
- L=[[“One”, “Two”],[“Three”, “Four”]]
- To access elements

  L[0][1]=Two

  L[1][1]=Four

To retrieve elements of a multidimensional array, you use multiple array access ([]) operators after the name of the top-level array. The first [] refers to the index of the top-level array. Subsequent array access operators refer to elements of nested arrays.

1.1.2 Multidimensional Arrays

P. language	Example
Java	int[][] myNumbers = { {2, 3, 4}, {5, 6, 7} }; To display the value, use System.out.println(myNumbers[1][2]);
C++	string letters[2][4] = { { "A", "B", "C", "D" }, { "E", "F", "G", "H" }}; cout << letters[0][2];
Python	myNumbers = [[2, 3, 4], [5, 6, 7]]; Print (myNumbers[1][2])

1.1. 3. **Associative Arrays**

3. Creating Associative Arrays \\Dictionary

- An associative array, which is like a **group objects**, is made of unordered keys and values.
- Associative arrays use keys instead of a numeric index to organize stored values. Each key is a unique string, and it's associated with and used to access one value.
- **That value can be a data type such as Number, Array, Object,** and so on. When you create code to find a **value that's associated with a key, you're indexing or performing a lookup**, which is the most typical reason for using an associative array.
- **For example,** if you had an array of names and ages that you needed to refer to a lot, you might use an associative array.
- Example 2: Define a series of properties of an object

1.1. 3. **Associative Arrays**

Ex1: **The students and their ages:**

- D={Mary:17,Paul:22, Joel:22,Isac:17, Qui:20}

Ex2: **Car attributes**

C={Version: Toyota, Color: Blue, origin: Uk, Date:1972}

To access the element we use

D[key]=value

To know the number of frequency of value, use incremental operators on specific value

Ex3: month format

Months ={ “Jan”:01,”Feb”:02,…”Dec”:12}

Ex4: Day format

Days ={“1st”:01, “2nd” :02, …,”28th’:28,…}

I-2: Operation

- Following are the basic operations supported by any array data type
- **Traverse** – print all the array elements one by one. [Slicing in array, by using For, while loop]
- **Insertion** – Adds an element at the given index.[append]
- **Deletion** – Deletes an element at the given index **\remove**
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

I-2: Operation

	Associated array create
Java	HashMap<String, Integer> Student_Age = new HashMap<>(); Example Map<String, int> Student_Age = new HashMap<>(); Student_Age.put(“Mary ", 17); Student_Age.put("Paul", 22);
C++	std::map<string, int> Student_Age{ { "Rithvik", 78 }, { “Mary", 17 }, { “Paul", 22 }, { “Isaac ", 22 }, { “Qui", 20 } };
Python	Student_Age={Mary:17,Paul:22, Joel:22,Isac:17, Qui:20}

I-2: Operation

	C++	Java	Python

Add	insert()
	student.insert(std::make_pair(3, Mary)
	// add elements with duplicate keys
	student_Age[3] = “Moise";
		put()
		Student_Age.put(“Eric”,20)
			Use operator []
			Student_Age[Eric]=20

Remove	erase() student_Age.erase(“Deny"); ;	remove() Student_Age.remove(2)	pop() Student_Age.pop(‘Eric’)
Find a key	find () iter = student_Age.find(2);	get() method to get value Q=Student_Age.get(1)	list() + keys() + index()
Remove	clear() Student_Age.clear();		clear()

I-2: Operation

	C++	Java	Python
Return values		values() Student_Age.values()	values() Student_Age.values()
Return Keys		.keySet() Student_Ag.keySet()	keys() Student_Age.keys()
Sort (Ascending)			
Sort (Descending)			

I-2: Operation +Algorithms

1. *Traversing-* It is used to **access each data** item exactly once so that it can be processed.

Operation	Syntax of algorithm
Given A=Array , N=length (A), T= Target element and i the value index from i=0,i=1,…,i=N	
Traversing	For i=0 to N Display A[i]

I-2: Operation +Algorithms

- Updating , Insertion algorithm syntax

Operation	Syntax
Update	Step 1: Start. Step2: Input (A=Array, pos=position of Old element, X=new element) Step 2: Set A[pos-1] = x. Step 3: Stop.
Insertion	1. Get the element value which needs to be inserted. 2. Get the position value. 3. Check whether the position value is valid or not. 4. If it is valid, Shift all the elements from the last index to position index by 1 position to the right. insert the new element in arr[position] 5. Otherwise, Invalid Position

I-2: Operation +Algorithms

- Delete operation /removing and search

Operation	Algorithm
Removing	1. Find the given element in the given array and note the index. 2. If the element found, Shift all the elements from index + 1 by 1 position to the left. Reduce the array size by 1. 3. Otherwise, print "Element Not Found"
Search	1. Iterate the array using the loop. 2. Check whether the given key present in the array i.e. arr[i] == key. 3. If yes, print "Search Found". 4. Else print "Search Not Found".

I-2: Operation

3.Inserting- It is used **to add a new data** item in

Insertion position	Java	C++	Python
At the end	-Use arrayList and add() function -Create insert function involves loop	Create insert function involves loop	Append()
At given index	Shifting elements to adjust the size of the array	Create insert function involves loop	Insert(idx, number)
Elongate two Arrays			extend ()

I-2: Operation

- *4. Deleting-* It is used **to delete an existing data** item from the given collection of data items.

Deletion	Java	C++	Python
At the end	Shifting the elements after that index to the left by one element.	Shift all the elements from index + 1 by 1 position to the left.	pop()
At given index	||	||	Remove()

I-2: Operation

4 Sorting- It is **used to arrange the data items** in some order i.e. in ascending or descending order in case of numerical data and in dictionary order in case of alphanumeric data.

Read algorithms used in sorting and search methods

I.3 Applications

- Below are some real-time applications of arrays.
- **Signal Processing:** Arrays are used in signal processing to represent a set of samples that are collected over time. This can be used in applications such as speech recognition, image processing, and radar systems.
- **Multimedia Applications:** Arrays are used in multimedia applications such as video and audio processing, where they are used to store the pixel or audio samples. For example, an array can be used to store the RGB values of an image.
- **Data Mining:** Arrays are used in data mining applications to represent large **datasets.** This allows for efficient data access and processing, which is important in real-time applications.
- **Robotics motion** : Arrays are used in robotics to represent the position and orientation of objects in 3D space. This can be used in applications such as motion planning and object recognition.
- **Chess game**

Cont’

- **Real-time Monitoring and Control Systems:** Arrays are used in real-time monitoring and control systems to store sensor data and control signals. This allows for real-time processing and decision-making, which is important in applications such as industrial automation and aerospace systems.
- **Financial Analysis:** Arrays are used in financial analysis to store historical stock prices and other financial data. This allows for efficient data access and analysis, which is important in real-time trading systems.
- **Scientific Computing:** Arrays are used in scientific computing to represent numerical data, such as measurements from experiments and simulations. This allows for efficient data processing and visualization, which is important in real-time scientific analysis and experimentation

UNITII: **LINEAR LIST**

2.1 Introduction

- A linear list is a list that displays the relationship of adjacency between elements.
- A Linear list can be defined as a data object whose instances are of the form (e1, e2, e3…en) where n is a finite natural number.
- The ei terms are the elements of the list and n is its length.
- The elements may be viewed as atomic as their individual structure is not relevant to the structure of the list. When n=0, the list is empty.
- When n>0,e1 is the first element and en the last. Ie;e1 comes before e2, e2 comes before e3 and so on.

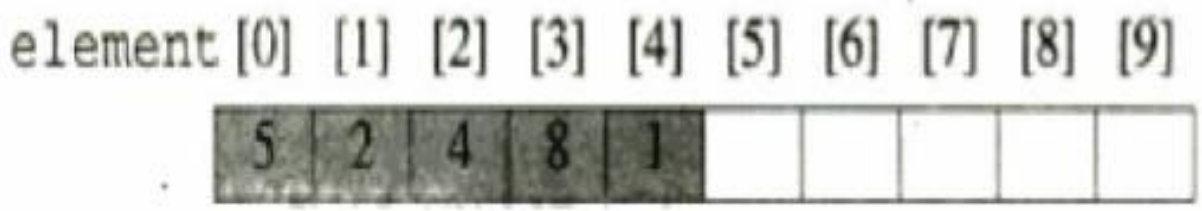
2.2 Operations

- The following are the operations that performed on the Linear List
- **Create** a Linear List
- **Destroy** a Linear List
- Determine whether the list is empty
- **Determine the size** of the List
- **Find the element** with a given index
- **Find the index** of a given number
- **Delete, erase or remove an element** given its index
- **Insert a new element** so that it has a given index
- Update

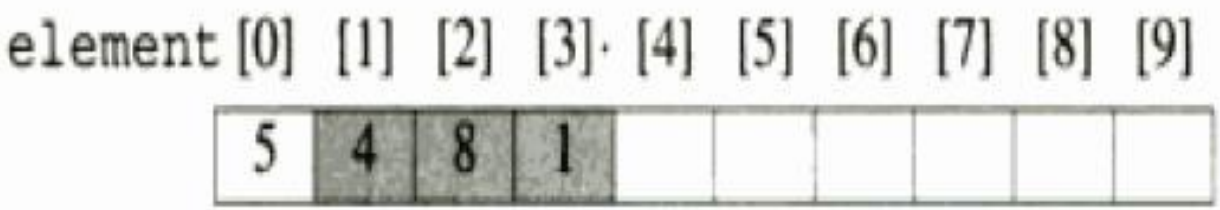
2.2.1 List Representation

- a formula based representation, a mathematical formula determines the location of each element. A simple mapping formulas is: **Location (i) = i-1**
- This equation states that the ith element of the list is in position i-1 of the array.

2.2.2 Insertion and Deletion

- 

(a) *location* (*i*)=*i*



(a) 2 removed from `element[1]`, `listSize` = 4

2.2.2 Insertion and Deletion

- **Insertion and Deletion of a Linear List:**
- Suppose we want to remove an element ei from the list by moving to its right down by 1.
- For example, to remove an element e1=2 from the list, we have to move the elements e2=4, e3=8,and e4=1,which are to the right of e1, to positions 1,2 and 3 of the array element. The below figure shows this result. The shaded elements are moved

2.2.2 OPERATION Function

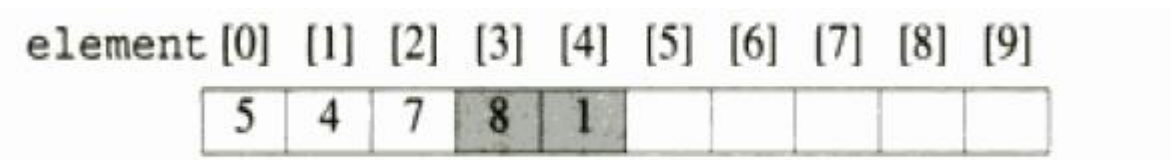
P.Languages	Example	Operation

C++	// an object of the List is created by the **name 'names'**
	//that will be used to store the nam es that will be entered by the user
	list<string> names;
		Add: push_back() Function

Java	Create a class object named ListClass ListClass object = new ListClass();	Add () Remove :remove(data) Update : object.update_data()
Python	MyList = list() **Mixed_data** = [20, ‘Emmy', 8.56, ‘a’]	Append() Remove ()

2.2.2 Insertion and Deletion

- To insert an element so that it becomes element I of a list, must move the existing element ei and all elements to its right one position right and then put the new element into position I of the array.
- For example to insert 7 as the second element of the list, we first move elements e2 and e3 to the right by 1 and then put 7 in to second position 2 of the array. The below figure shows this result. The shaded elements were moved.



(b) 7 inserted at element [2], listSize = 5

2.2.3 Applications

- List of viewers on Facebook
- List of students in the class
- Sorting algorithms (Lists are essential in sorting algorithms such as quick sort and merge sort.
- An alphabetized list of students in a class
- A list of exam scores in non decreasing order
- A list of gold medal winners in the Olympics
- An alphabetized list of members of Congress

2.2.4 Practices

Practical exercises will be on:

- ✓ Creating list
- ✓ Traversing
- ✓ Update
- ✓ Replace
- ✓ And so on [See on exercise flowchart, pseudo codes and writing programs]

Unit III: **Linked list Representation And Chains**

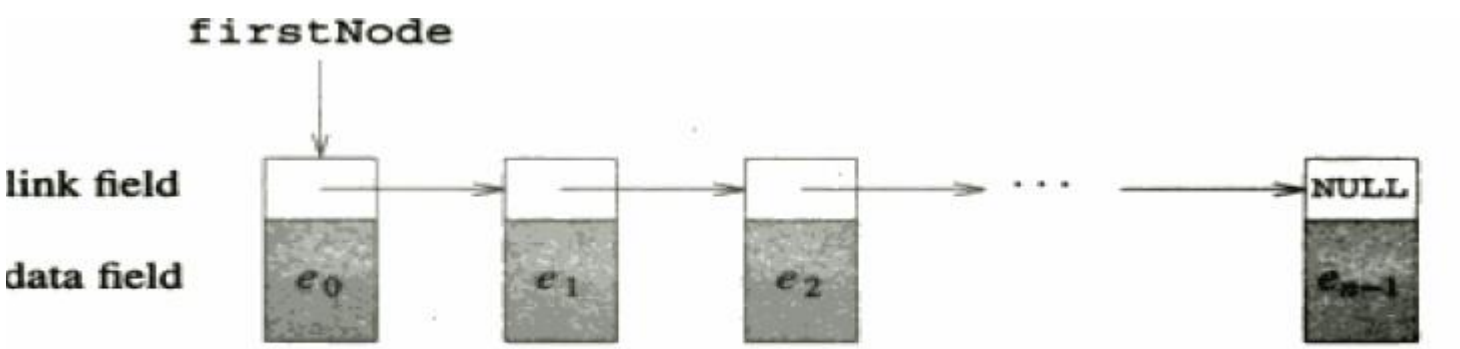
- 3.1. Introduction
- In a linked list representation each element of an instance of a data object is represented in a **cell or node**. **The element are not in contiguous location. A={[3,4],[5, 9 ,10]}**
- **The nodes** however need not be component of an array and no formula is used to locate individual elements.
- Instead of each node keeps explicit information about the location of other relevant nodes. This explicit information about the location of another node is called Link or Pointer.

3.1. Introduction

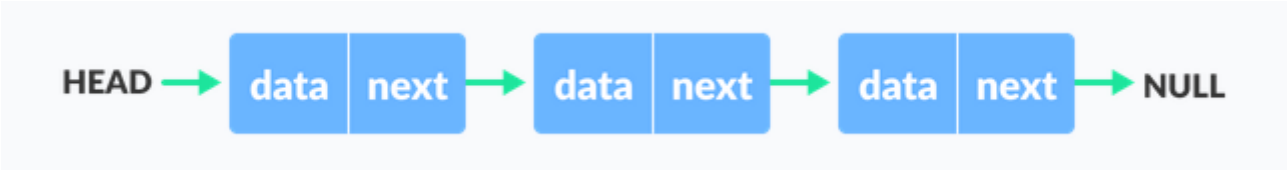
- Let L=(e1, e2, e3…en) be a linear List. In one possible linked representation for this list, each element ei is represented in a separate node. Each node has exactly one link field that is used to locate the next element in the linear list. So the node for ei links to that for ei+1, 0<=i<n-1.
- The node for **en-1** has no need to link to and so its link field is **NULL**.

3.1. Introduction

- The pointer variables first locate the first node in the representation. The below figure shows the linked representation of a List=(e1, e2, e3…en).



Linked representation of a linear list



3.1. Introduction

- Since each node in the Linked representation of the **above figure has exactly one link,** the structure of this figure is called a '**Single Linked List'**.
- The nodes are ordered from left to right with each node (other than last one) **linking to the next**, and
- the last node has a **NULL link**, the structure is also called a **chain.**

3.1. Introduction

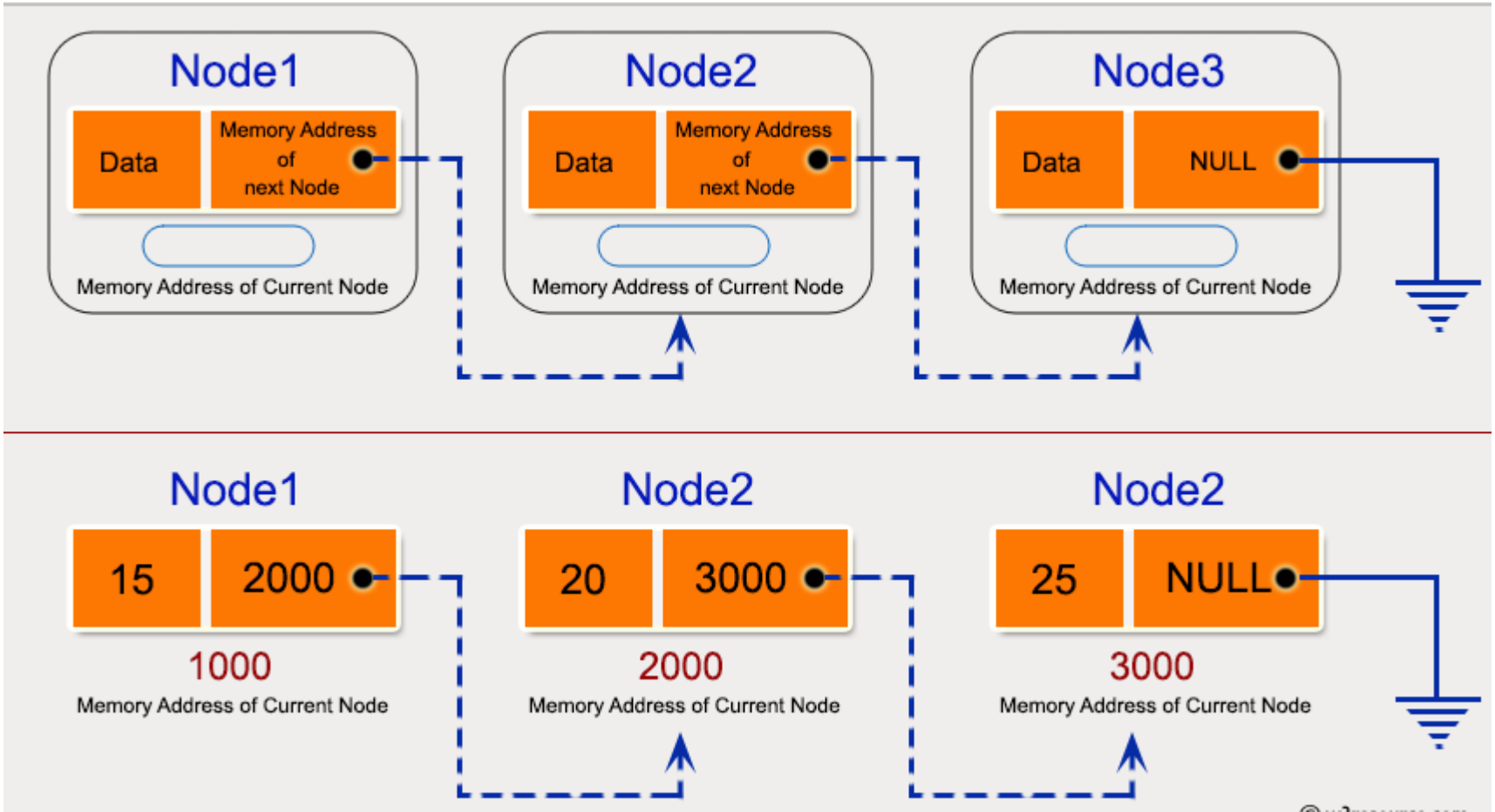
- We have three categories of linked list
- Single list
- Double linked list
- Circular linked list

3.2 single list Operations

- Some operations are performed in linked list
- insertion: Add an item to the linked list at the head of the list.
- Search Find an item within the linked list.
- Deletion : Remove a given item with a given value.
- Check the existence of value: Returns whether the linked list is empty or not.
- Counting : Returns the number of items in the linked list.

3.2 single list presentation

- The **Visual Presentation:**

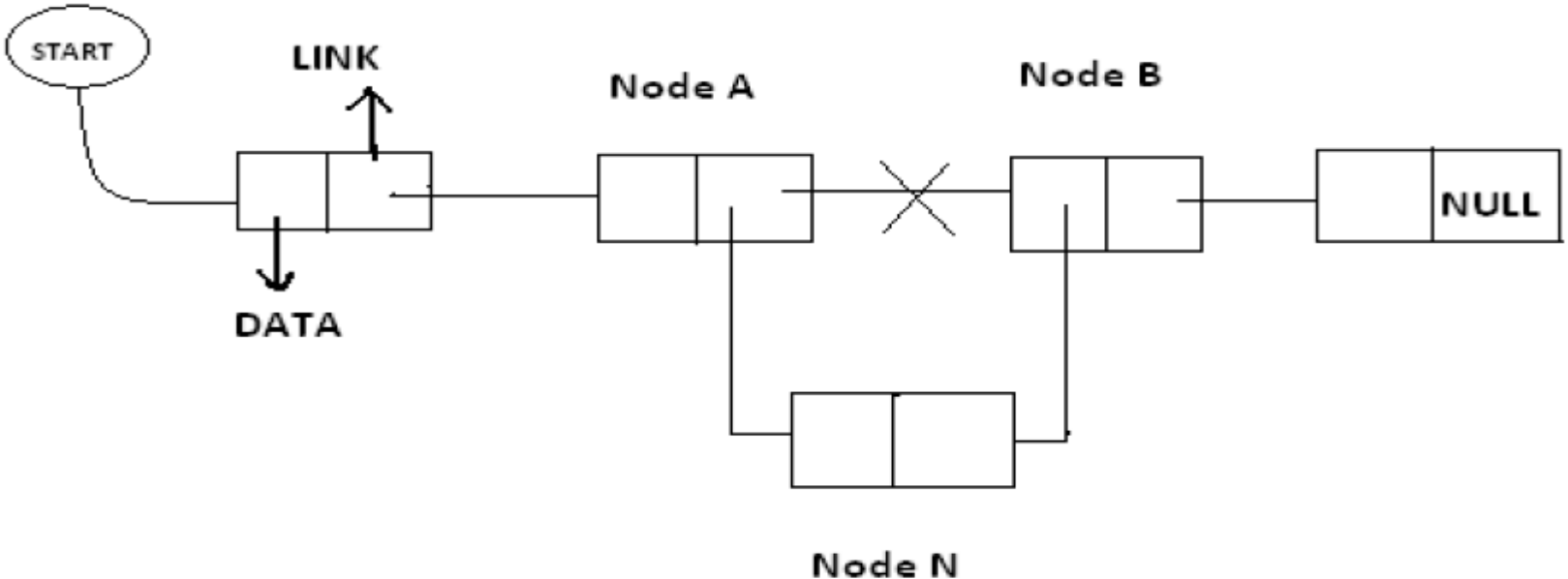


3.2 single list creation steps

- Initialize node class
- Allocate memory
- Assign data value
- Connect nodes
- Save address od first node in head

3.2.1: Insertion

- Let the list be a Linked list with successive nodes A and B as shown in below figure. Suppose a node N id to be inserted into the list between the node A and B.



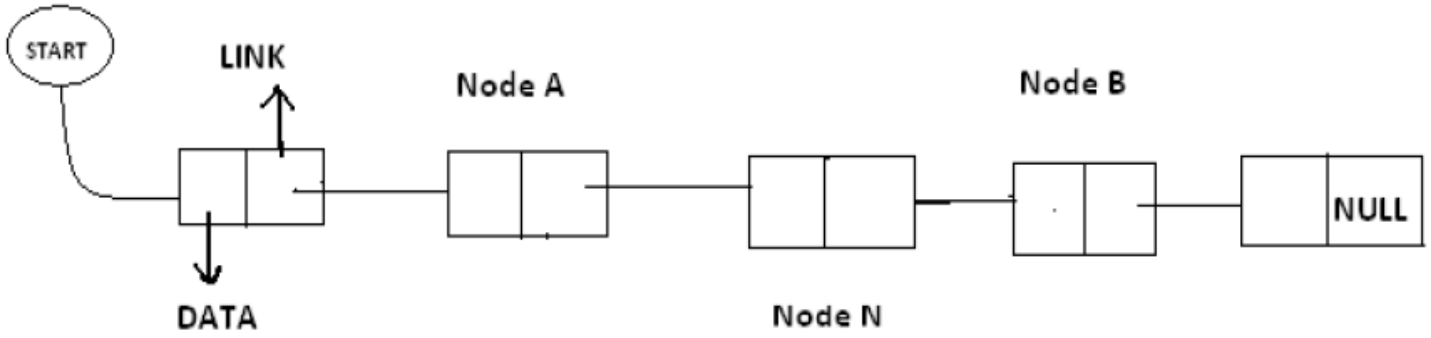
3.2.1: Insertion

- Simply, the new node start by point its link to its right node, and then its data to left node
- The syntax

```
NewNode.next -> rightNode;

LeftNode.next -> NewNode ;
```

- After linking them , the graph becomes



3.2.1.1 Insertion at Beginning

- **Insertion at Beginning //Algorithm**
- We don't connect the node at both left and right. Nothing exits.

1. START
2. Create a node to store the data
3. Check if the list is empty
4. If the list is empty, add the data to the node and assign the head pointer to it.
5. If the list is not empty, add the data to a node and link to the current head. Assign the head to the newly added node.
6. END

3.2.1.1 Insertion at Beginning

- The syntax of creating a class of node (data and pointer)`

C or C++	Java	Python

struct node {
int data;
struct node *next;};
	node (int value) {
	data = value;
	next = null; }
		Class Node
		def __init__(self):
		self.data = data
		self.next = None

- Create another class of linked list that contains [initial header, display method, and so on..

3.2.1.2 Insertion at Ending

- In this operation, we are adding an element at the ending of the list.
- **Algorithm**

1. **START**
2. **Create a new node and assign the data**
3. **Find the last node**
4. **Point the last node to new node**
5. **END**

3.2.1.3 Insertion at a Given Position

- Algorithm

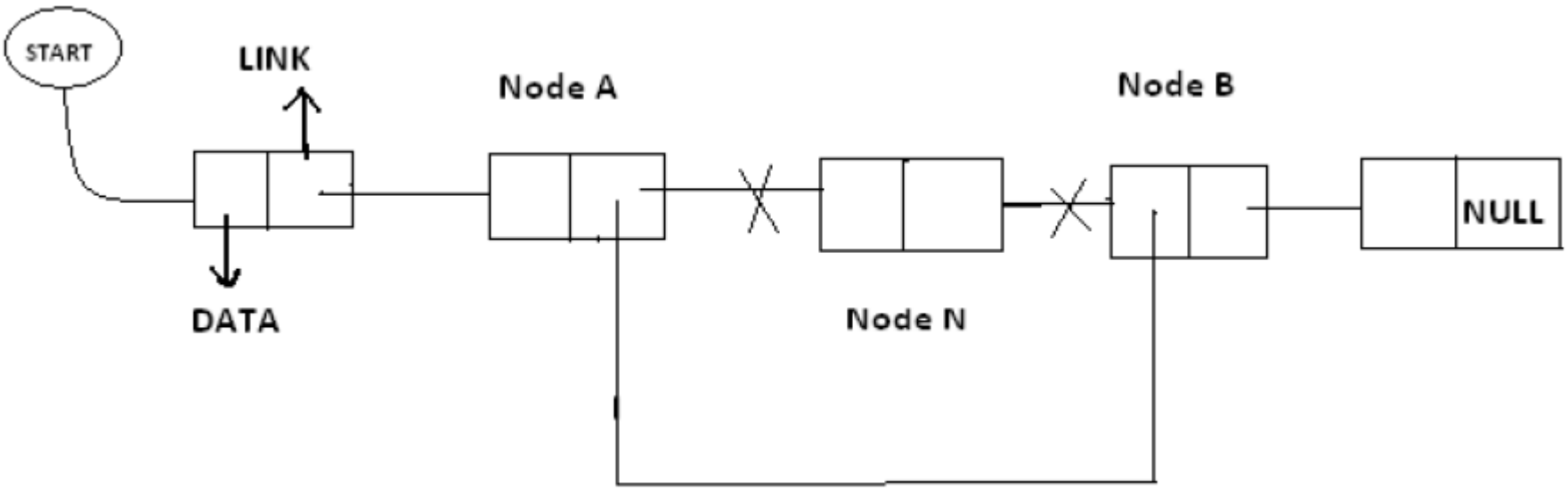
1. START
2. Create a new node and assign data to it
3. Iterate until the node at position is found
4. Point first to new first node
5. END

- Syntax

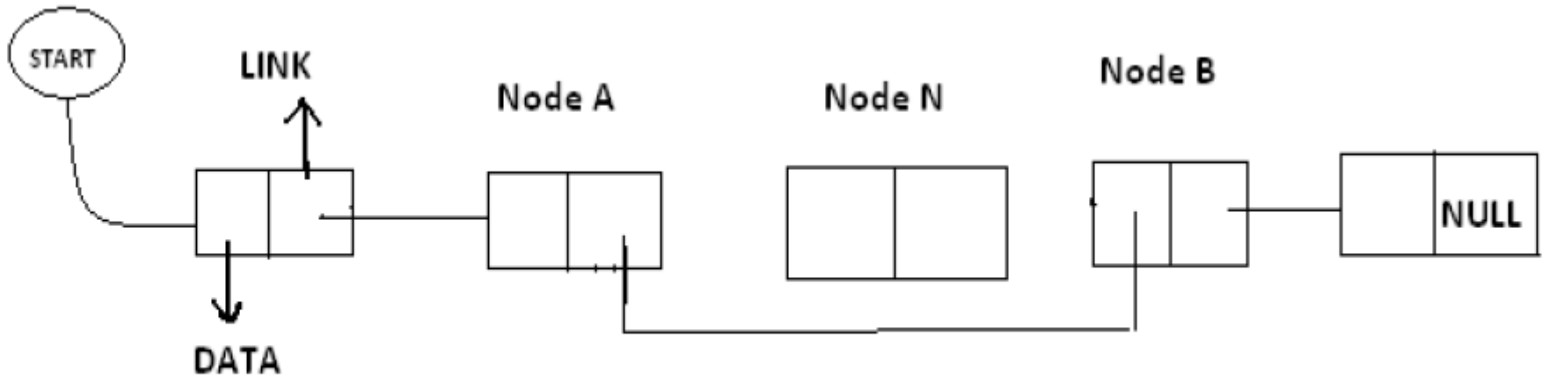
```
NewNode = Node(newdata)
 NewNode.next = nodeatpos.next
nodeatpos.next = NewNode
```

3.2.2 Deletion

- Let list be a Linked list with node N between Nodes A and B is as shown in the following figure.



3.2.2 Deletion

- 

After Deletion Node N in Between Node A abd Node B

- In the new list the node N is to be deleted from the Linked List. The deletion occurs as the link field in the Node A is made to point node B this excluding node N from its path.

3.2.2.1 Deletion syntax

- Deletion in linked lists is also performed in three different ways. They are as follows
- **Deletion at Beginning**
- **Deletion at Ending**
- **Deletion at a Given Position**

A. Deletion at Beginning

- The algorithm

1. **START**
2. **Assign the head pointer to the next node in the list**
3. **END**

B Deletion at Ending

- In this deletion operation of the linked, we are deleting an element from the ending of the list.

1. START
2. Iterate until you find the second last element in the list
3. Assign NULL to the second last element in the list. 4. END

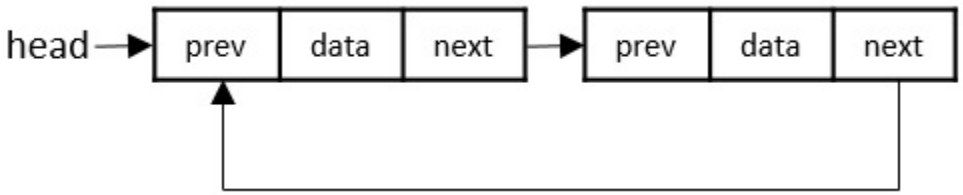
C-Deletion at a Given Position

- In this deletion operation of the linked, we are deleting an element at any position of the list.
- **Algorithm**

1. START
2. Iterate until find the current node at position in the list.
3. Assign the adjacent node of current node in the list to its previous node.
4. END

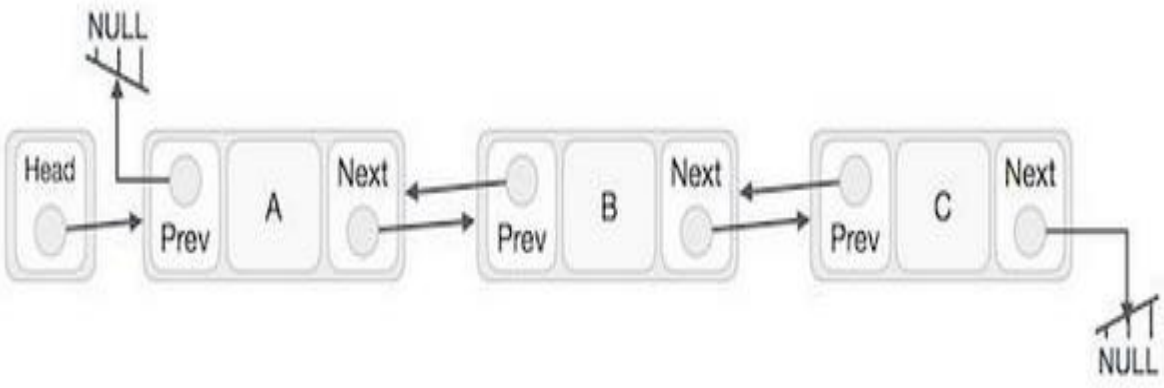
3.3.3 DOUBLE LINKED LIST

- In certain applications it is very desirable that list be traversed **in either forward direction or Back word direction**.
- The links are used to denote the preceding and succeeding of the node. The link denoting the preceding of a node is called **Left Link.** The link denoting succeeding of a node is called Right Link.
- The list contain this type of node is called a "**Double Linked List**" or "**Two Way List**". The Node structure in the Double Linked List is as follows:
- The property of **Double Linked List implies that each node must contain two link fields instead of one**

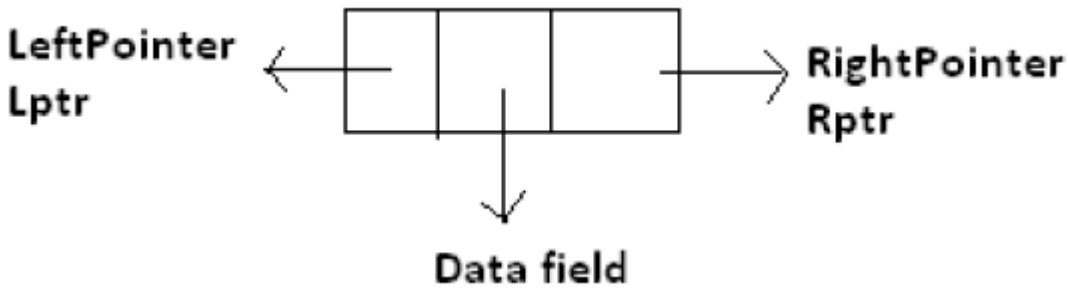


Cont’

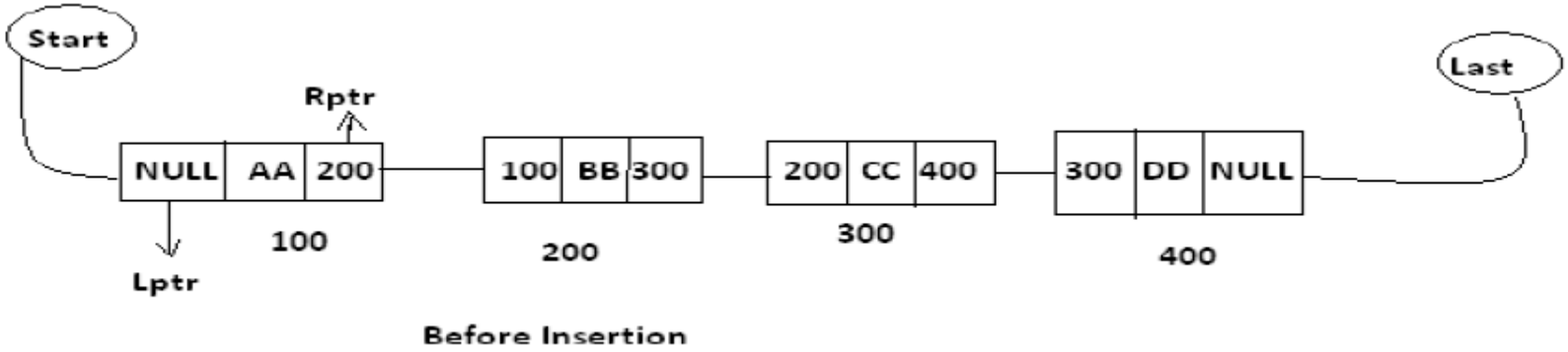
- Doubly Linked List contains a link element called first and last.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Each link is linked with its previous link using its previous link.
- The last link carries a link as null to mark the end of the list.



Cont’

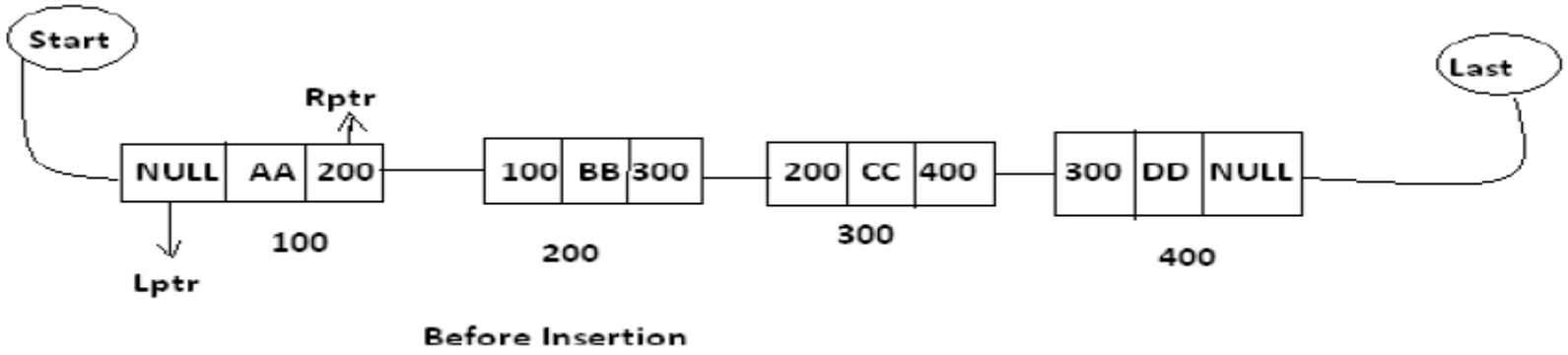


- **Lptr** contains the address memory of left node node. **Rptr** contains the memory address of next node. Data Contains the Linked List is as follows.



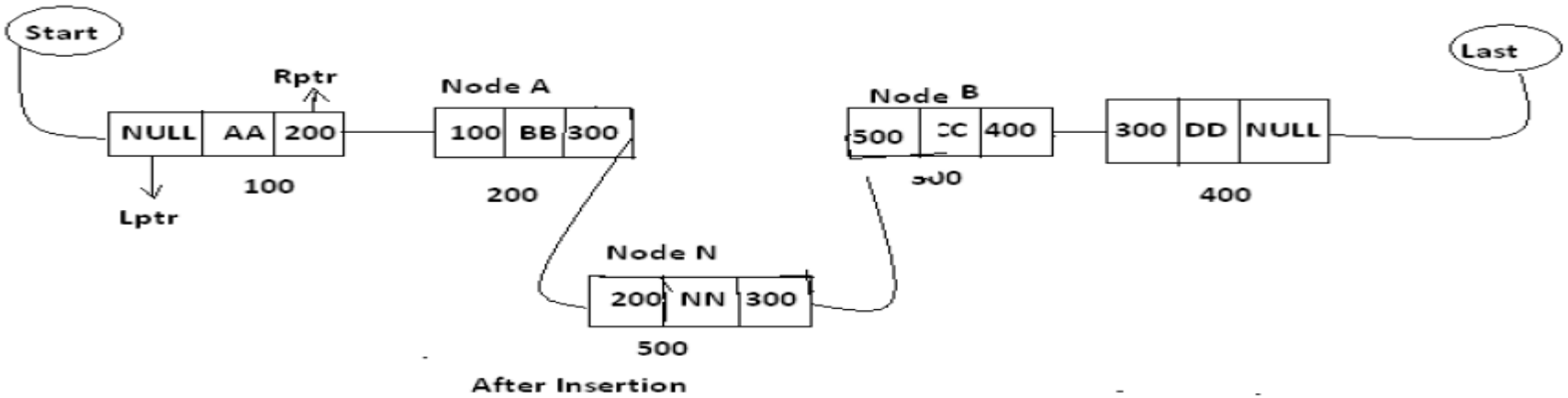
Insertion in to the Double Linked List:

- Let list be a double linked list with successive modes A and B as shown in the following diagram. Suppose a node N is to be inserted into the list between the node s A and B this is shown in the following diagram.



Before Insertion

Con't



- As in the new list the right pointer of node A points to the new node N ,the Lptr of the node 'N' points to the node A and Rptr of node 'N' points to the node 'B' and Lpts of node B points the new node 'N'.

Insertion at the Beginning

- In this operation, we create a new node with three compartments, one containing the data, the others containing the address of its previous and next nodes in the list. This new node is inserted at the beginning of the list
- The algorithm

1. **START**
2. **Create a new node with three variables: prev, data, next.**
3. **Store the new data in the data variable**
4. **If the list is empty, make the new node as head.**
5. **Otherwise, link the address of the existing first node to the next variable of the new node, and assign null to the prev variable.**
6. **Point the head to the new node.**
7. **END**

Insertion at the End

- In this insertion operation, the new input node is added at the end of the doubly linked list; if the list is not empty. The head will be pointed to the new node, if the list is empty.
- Algorithm

1. **START**
2. **If the list is empty, add the node to the list and point the head to it.**
3. **If the list is not empty, find the last node of the list.**
4. **Create a link between the last node in the list and the new node.**
5. **The new node will point to NULL as it is the new last node.**
6. **END**

Deletion at the Beginning

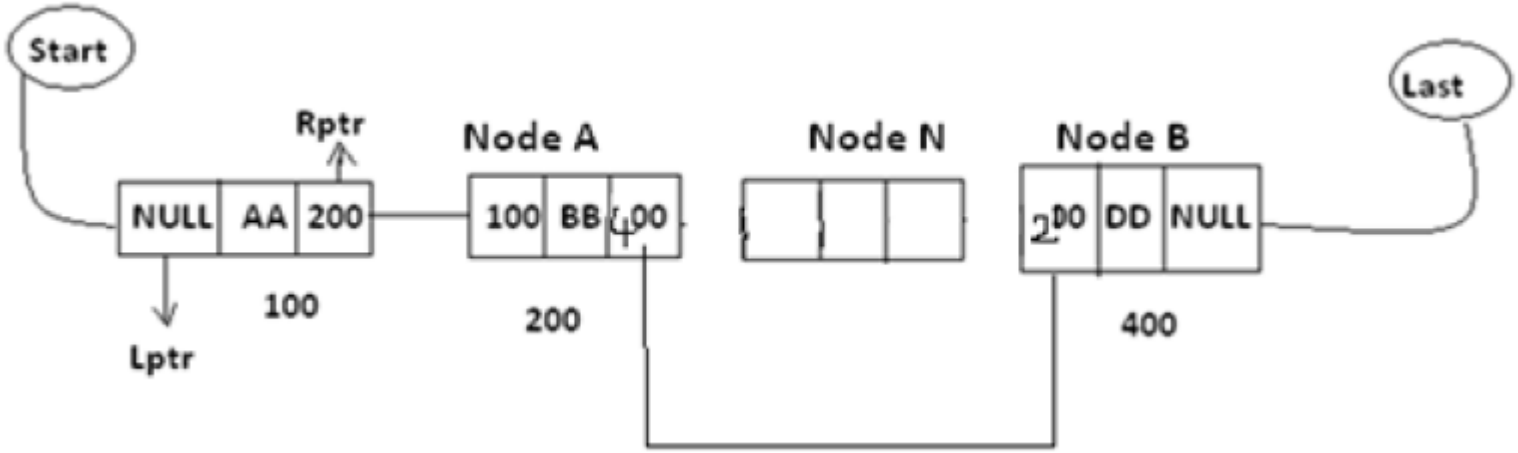
- This deletion operation deletes the existing first nodes in the doubly linked list. The head is shifted to the next node and the link is removed.

Algorithm

1. **START**
2. **Check the status of the doubly linked list**
3. **If the list is empty, deletion is not possible**
4. **If the list is not empty, the head pointer is shifted to the next node**
5. **END**

Deletion Of Double Linked List

1. Let list be a linked list contains node N between the nodes A and B as shown in the following diagram.

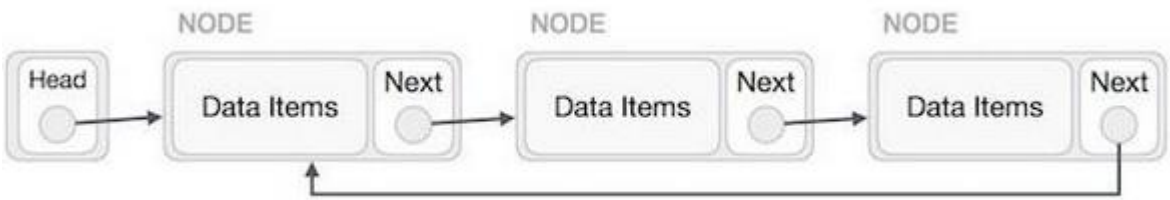


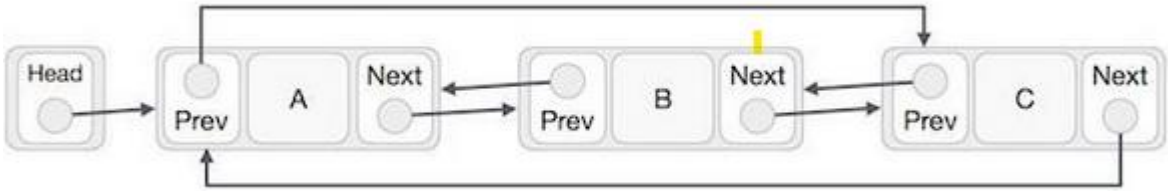
Cont’

- Support node N is to be deleted from the list diagram will appear as the above mention double linked list.
- The deletion occurs as soon as the right pointer field of node A charged, so that it points to node B and the lift point field of node B is changed. So that it pointes to node A.

3.3.4 Circular Linked List:-

- **Circular Linked List** is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list.
- **link fields of last node of the list contains the address of the first node of the list instead of contains a null pointer**





Cont’

- ***Advantages:-*** Circular list are frequency used instead of ordinary linked list because in circular list all nodes contain a valid address. The important feature of circular list is as follows.
- (1) In a circular list every node is accessible from a given node.
- (2) Certain operations like concatenation and splitting becomes more efficient in circular list.
- **Disadvantages:** Without some conditions in processing it is possible to get into an infinite Loop.

Insertion Operation in CLL

- Algorithm

1. **START**
2. **Check if the list is empty**
3. **If the list is empty, add the node and point the head to this node**
4. **If the list is not empty, link the existing head as the next node to the new node.**
5. **Make the new node as the new head**
6. **END**

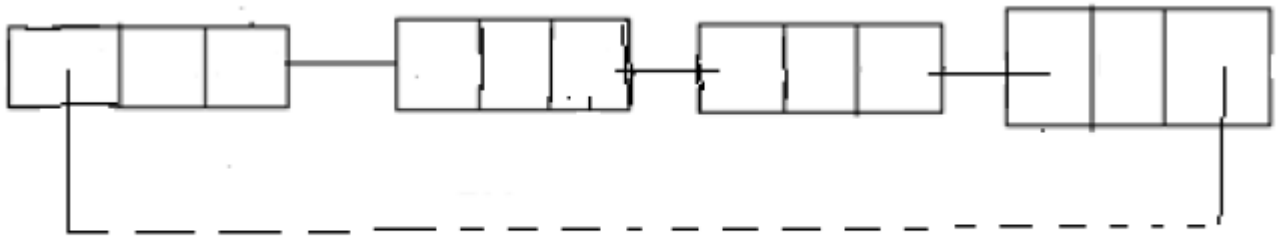
Deletion Operation in CLL

- Algorithm

1. **START**
2. **If the list is empty, then the program is returned.**
3. **If the list is not empty, we traverse the list using a current pointer that is set to the head pointer and create another pointer previous that points to the last node**
4. **Suppose the list has only one node, the node is deleted by setting the head pointer to NULL**
5. **If the list has more than one node and the first node is to be deleted, the head is set to the next node and the previous is linked to the new head**
6. **If the node to be deleted is the last node, link the preceding node of the last node to head node.**
7. **If the node is neither first nor last, remove the node by linking its preceding node to its succeeding node**
8. **END**

Circular Double Linked List

- The rpt field of the **last node of the list contain the address of the first node** add the left points of the first node contains the address of the last node of the list instead of containing null pointer.



Circular Double Linked List

Difference between single linked list and double linked list?

Single linked list(SLL)

1.In Single Linked List the list will be traversed in only one way ie; in forward.

2. In Single Linked List the node contains one link field only.

3. Every node contains the address of next node.

4.The node structure in Single linked list is as follows:

Double linked list(DLL)

1. In Double Linked List the list will be traversed in two way ie; either forward and backward

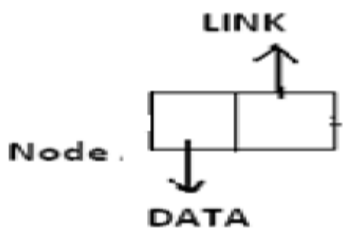
2. In Double Linked List the node contains two link fields.

3. Every node contains the address of next node as well as preceding node.

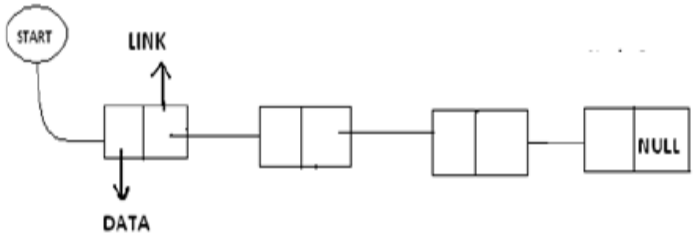
4.the node structure in double linked list is as follows:

Cont’

Single link

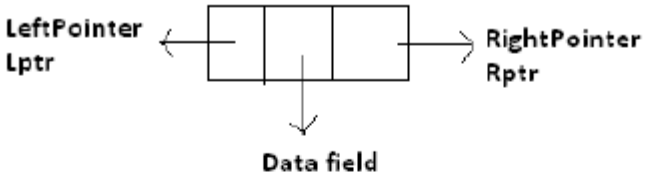


5. The conceptual view of SLL is as follows:

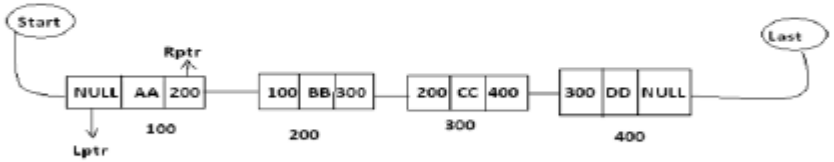


6. SLL are maintained in memory by using two arrays.

Double link



5.the conceptual view of DLL is as follows:



6. DLL is maintained in memory by using three arrays.

Difference between Linear List and Linked List?

Arrays /linear/sequential

1. Arrays are used in the predictable storage requirement ie; exert amount of data storage required by the program can be determined.

2. In arrays the operations such as insertion and deletion are done in an inefficient manner.

3. The insertion and deletion are done by moving the elements either up or down.

4. Successive elements occupy adjacent space on memory.

5. In arrays each location contain DATA only

Linked list/allocation

1. Linked List are used in the unpredictable storage requirement ie; exert amount of data storage required by the program can't be determined.

2. In Linked List the operations such as insertion and deletion are done more efficient manner ie; only by changing the pointer.

3. The insertion and deletion are done by only changing the pointers.

4. Successive elements need not occupy adjacent space.

5. In linked list each location contains data and pointer to denote whether the next element present in the memory.

Cont’

Linear

6. The linear relation ship between the data elements of an array is reflected by the physical relation ship of data in the memory.

7. In array declaration a block of memory space is required.

8.There is no need of storage of pointer or lines

linked

6. The linear relation ship between the data elements of a Linked List is reflected by the Linked field of the node.

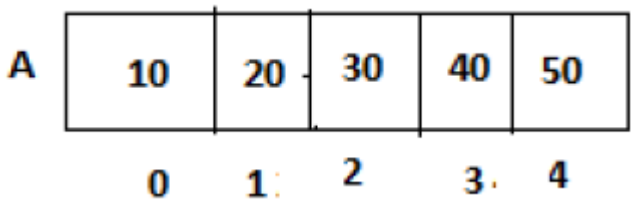
7. In Linked list there is no need of such thing.

8. In Linked list a pointer is stored along into the element.

Cont’

Array/list

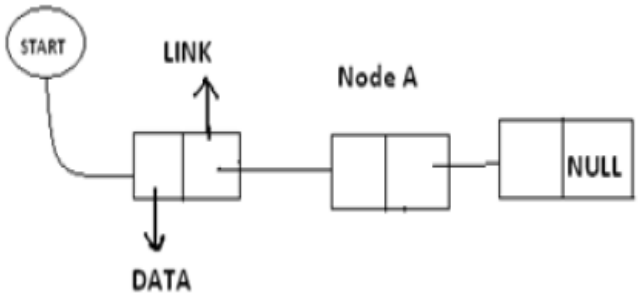
9.The Conceptual view of an Array is as follows:



10.In array there is no need for an element to specify whether the next is stored

Linked

9. The Conceptual view of Linked list is as follows:



10.In array there is no need for an element to specify whether the next is stored

3.3 Application

- **Image viewer** – **Previous** and **next** images are linked and can be accessed by the next and previous buttons.
- **Previous and next page in a web browser** – We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.
- **Music Player Songs** in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.
- **GPS navigation systems** Linked lists can be used to store and manage a list of locations and routes, allowing users to easily navigate to their desired destination.

3.3 Application

- **Task Scheduling-** Operating systems use linked lists to manage task scheduling, where each process waiting to be executed is represented as a node in the list.
- **File Systems-** File systems use linked lists to represent the hierarchical structure of directories, where each directory or file is represented as a node in the list.
- **Undo/Redo Functionality-** Many software applications implement undo/redo functionality using linked lists, where each action that can be undone is represented as a node in a doubly linked list.

3.3 Application

- Circular linked lists can be used in networking. For instance, to implement circular buffers for streaming data, such as video and audio, in networking applications.
- Video games use circular linked lists to manage sprite animations.

QuizNo2

- Use the digits of your registration number to create a double linked list .Use memory address of the next field of current node and the memory address of the node.

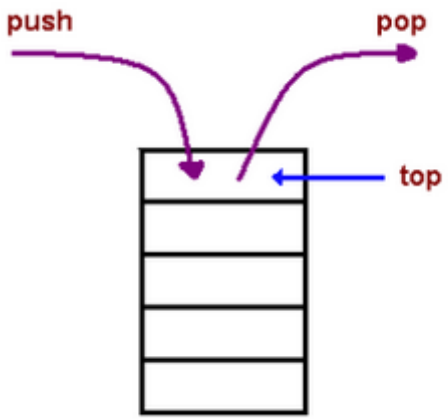
UNIT IV: STACK

4.1: Introduction:

- A Stack is linear data structure. A stack is a list of elements in which an element may be inserted or deleted only at one end, called the **top of the stack**. Stack principle is **LIFO (last in, first out)**. Which element inserted last on to the stack that element deleted first from the stack.
- As the items can be added or removed only from the top i.e. the last item to be added to a stack is the first item to be removed.

Cont’

Stack



Example of stack :

The simplest application of a **stack is to reverse a word**. You push a given word to stack – letter by letter – and then pop letters from the stack

Cont’

- Example 2

Stack of books

Stack of Plates

Stack of Toys

4.2. Operations on stack

- The two basic operations associated with stacks are:
  1. **Push** /append
  2. Pop

While performing push and pop operations the following test must be conducted on the stack.

a) Stack is empty or not

b) stack is full or not

1. Push/Append : Push operation is used to add **new elements in to the stack**. At the time of addition first check **the stack is full or not**. If the stack is full it generates an error message "stack overflow".

2. Pop: Pop operation is used to delete elements from the stack. At the time of deletion first check the stack is empty or not. If the stack is empty it generates an error message "stack underflow".

CON’

- Summary, the Operation that could be performed on stack

push() – adds an item to stack
pop() – removes the top stack item
peek() – gives value of top item without removing it
isempty() – checks if stack is empty
isfull() – checks if stack is full

Representation of Stack :

- The stack should be represented in two ways:
  1. Stack using array
  2. Stack using linked list

Stack using array:

- Let us consider a stack with 6 elements capacity. This is called as the size of the stack. The number of elements to be added should not exceed the maximum size of the stack. If we attempt to add new element beyond the maximum size, we will encounter a ***stack overflow*** condition.
- Similarly, you cannot remove elements beyond the base of the stack. If such is the case, we will reach a ***stack underflow*** condition.

1. Insert an element

- When an element is added to a stack, the operation is performed by push() or append. Below Figure shows the creation of a stack and addition of elements using

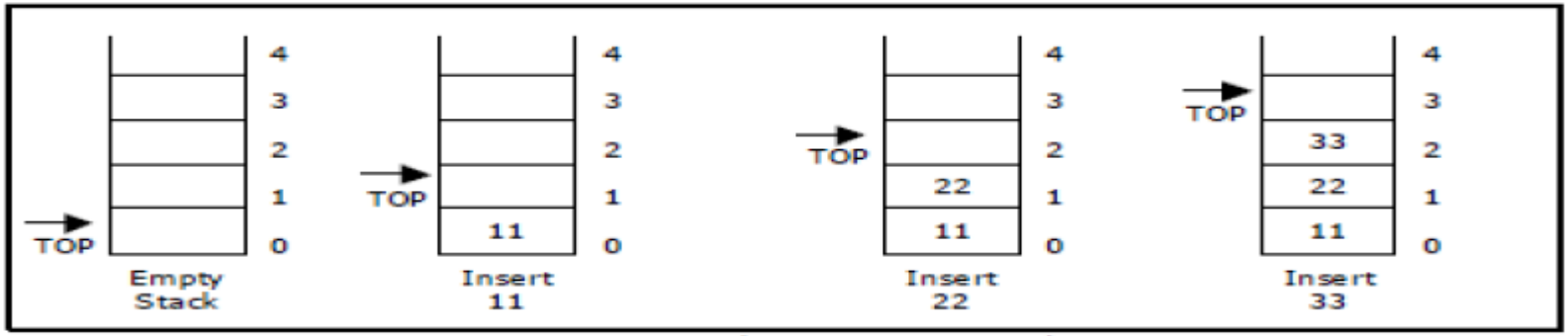


Figure . Push operations on stack

- Initially **top=-1**, we can insert an element in to the stack, increment the top value i.e **top=top+1**. We can insert an element in to the stack first check the condition is stack is full or not. i.e **top>=size-1**. Otherwise add the element in to the stack.

Delete an element

- When an element is taken off from the stack, the operation is performed by pop(). Below figure shows a stack initially with three elements and shows the deletion of elements using pop().

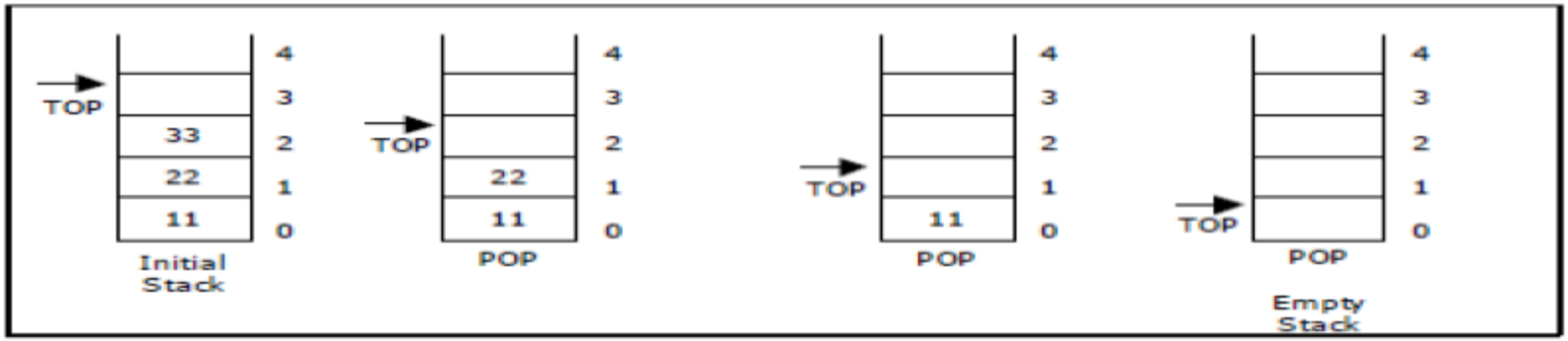
- 

Figure Pop operations on stack

Cont’

- We can insert an element from the stack, decrement the top value i.e **top=top-1**.
- We can delete an element from the stack first check the condition is stack is empty or not.
- i.e **top==-1**. Otherwise remove the element from the stack.

Syntax in	INSERTION	DELETION
C++	push()	pop()
Java	push(a)	pop()
Python	append (a)	pop()

Display an element

- This operation performed display the elements in the stack. We display the element in the stack check the condition is stack is empty or not i.e top==-1.Otherwise display the list of elements in the stack.

Syntax in	Example
Java	System.out.println
C++	cout
python	Print()

Syntax

- Initialize stack=[] ,
- int top=-1
- Int max_size =val;
- IsEmpty iff top=Val;
- Isfull ifi top=max_size

2. Stack using Linked List:

- We can represent a stack as a linked list. In a stack push and pop operations are performed at one end called top. We can perform similar operations at one end of list using top pointer. The linked stack looks as shown in figure.

Cont’

- d

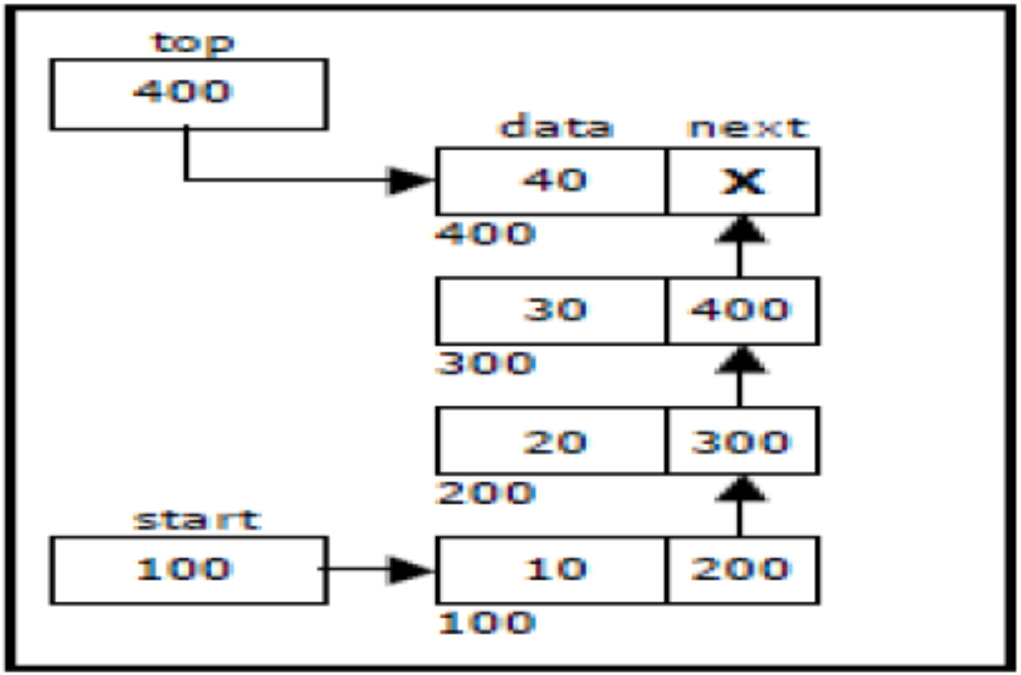


Figure Linked stack representation

4.3 Applications of stack:

1. Stack is used by compilers to **check for balancing of parentheses, brackets and braces**.

2. Stack is used to evaluate a infix, **postfix expression**.

3. Stack is used to convert an **infix expression** into postfix/prefix form.

4. In recursion, all intermediate arguments and return values are stored on the processor's stack.

Infix, Prefix and postfix

- An algebraic expression can be represented using **three different notations.** They are infix, postfix and prefix notations:
- **Infix:** It is the form of an arithmetic expression in which we fix (place) the **arithmetic operator in** between the two operands.

Example: A + B

- **Prefix:** It is the form of an arithmetic notation in which we fix (place) the arithmetic operator before (pre) its two operands. The prefix notation is called as polish notation.

Example: + A B

- **Postfix:** It is the form of an arithmetic expression in which we fix (place) the arithmetic operator after (post) its two operands. The postfix notation is called as *suffix notation* and is also referred to *reverse polish notation*.

Example: A B +

UNIT V: **QUEUE**

- 5.1 Introduction
- A queue is linear data structure and collection of elements. A queue is another special kind of list, where items are inserted at one end called the **rear** and deleted at the other end called the **front**. The principle of queue is a **"FIFO" or "First-in-first-out"**.
- Queue is an abstract data structure. A queue is a useful data structure in programming. **It is similar to the ticket queue outside a cinema hall**, where the first person entering the queue is the first person who gets the ticket.
- A real-world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first.

Cont’

-
LAST IN
LAST OUT
ONE WAY
FIRST IN
FIRST OUT

Rear of
Queue
Front of
Queue
next...
PAY HERE

Cont’

- The operations for a queue are analogues to those for a stack; the difference is that the insertions go at the end of the list, rather than the beginning.
- Types of Queue:
- Priority queue, circular Queue,

5.2 Operations on QUEUE:

- A queue is an object or more specifically an abstract data structure (ADT) that allows the following operations:
- **Enqueue or insertion:** which inserts an element at the end of the queue.
- **Dequeue or deletion:** which deletes an element at the start of the queue.

The below operations can be performed on a stack –
enqueue() – adds an item to rear of the queue
dequeue() – removes the item from front of the queue
peek() – gives value of front item without removing it
isempty() – checks if Queue is empty
isfull() – checks if Queus is full

INSERTION

- To add element on Queue we use the function like push, add or append

Synthax in	Example
C++	push()
Java	add()
Python	append ()

DELETION

- It means that you must pick the 1st element

i.e the element at position 0 .We use the key word pull or pop(0)

Syntax in	Deletion
C++	
Java	remove()
Python	pop(0)

Queue operations

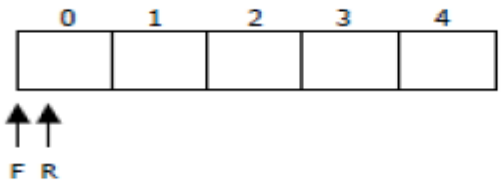
1. Two pointers called FRONT and REAR are used to keep track of the **first and last el**ements in the queue.
2. When initializing the queue, we set the value of FRONT and REAR to 0.
3. On **enqueing** an element, we increase the value of REAR index and place the new element in the position pointed to by REAR.
4. On **dequeueing** an element, we return the value pointed to by FRONT and increase the FRONT index.
5. Before **enqueing**, we check if queue is already full.
6. Before **dequeuing**, we check if queue is already empty.
7. When enqueing the first element, we set the value of FRONT to 1.
8. When dequeing the last element, we reset the values of FRONT and REAR to 0.

Representation of Queue

- The queue can be represented in two ways:
- 1. Queue using Array
- 2. Queue using Linked List

1.Queue using Array:

Let us consider a queue, which can hold maximum of five elements. Initially the queue is empty.

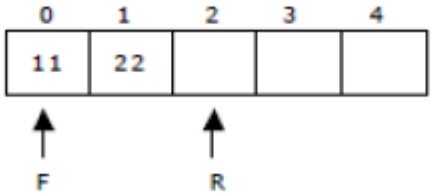


Queue Empty
FRONT = REAR = 0

Now, insert 11 to the queue. Then queue status will be:

REAR = REAR + 1 = 1
FRONT = 0

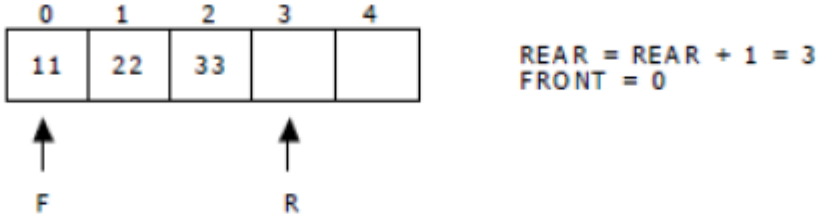
Next, insert 22 to the queue. Then the queue status is:



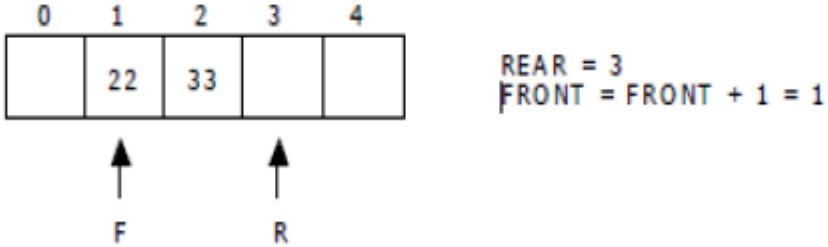
REAR = REAR + 1 = 2
FRONT = 0

Cont’

- Again insert another element 33 to the queue. The status of the queue is:
- Now.



Now, delete an element. The element deleted is the element at the front of the queue.So the status of the queue is:

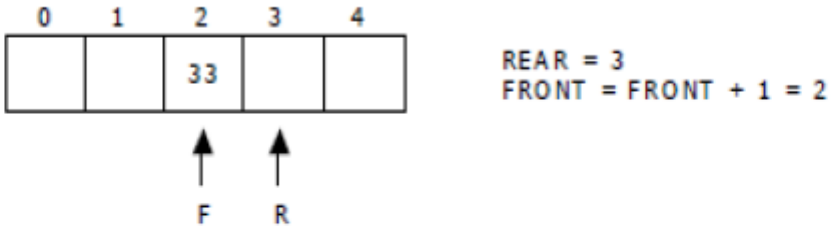


Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The queue status is as follows:

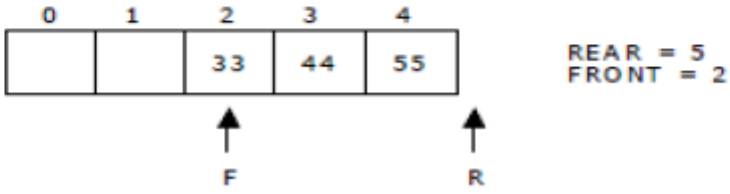
Cont’

- C

Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The queue status is as follows:

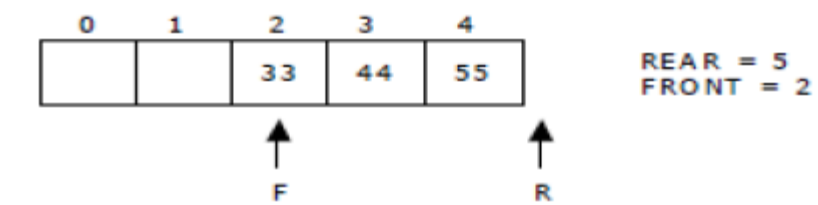


Now, insert new elements 44 and 55 into the queue. The queue status is:



Cont’

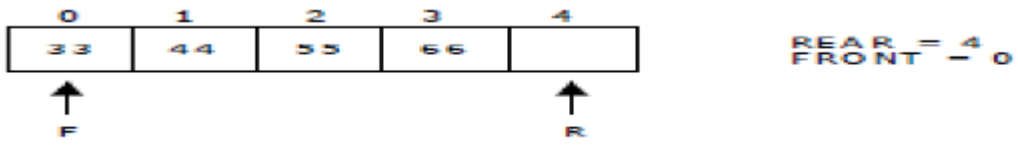
- Next insert another element, say 66 to the queue.
- We cannot insert 66 to the queue as the rear crossed the maximum size of the queue (i.e., 5). There will be queue full signal. The queue status is as follows:



Cont’

- Now it is not possible to insert an element 66 even though there are two vacant positions in the linear queue. To overcome this problem the elements of the queue are to be shifted towards the beginning of the queue so that it creates vacant position at the rear end. Then the FRONT and REAR are to be adjusted properly. The element 66 can be inserted at the rear end. After this operation, the queue status is as follows:

Cont’



- This difficulty can overcome if we treat queue position with index 0 as a position that comes after position with index 4 i.e., we treat the queue as a **circular queue.** d

Syntax

- Initialize Q=[]
- Int max_size=Val;
- Int front =-1
- Int back =-1
- isEmpty iff front=-1
- IsFull ifif back=max_size;
- Add element Q[i]=item, loop and back++

Queue using Linked list:

- We can represent a queue as a linked list. In a queue data is deleted from the front end and inserted at the rear end. We can perform similar operations on the two ends of a list. We use two pointers *front* and *rear* for our linked queue implementation.
- The linked queue looks as shown in figure:

Cont’

- c

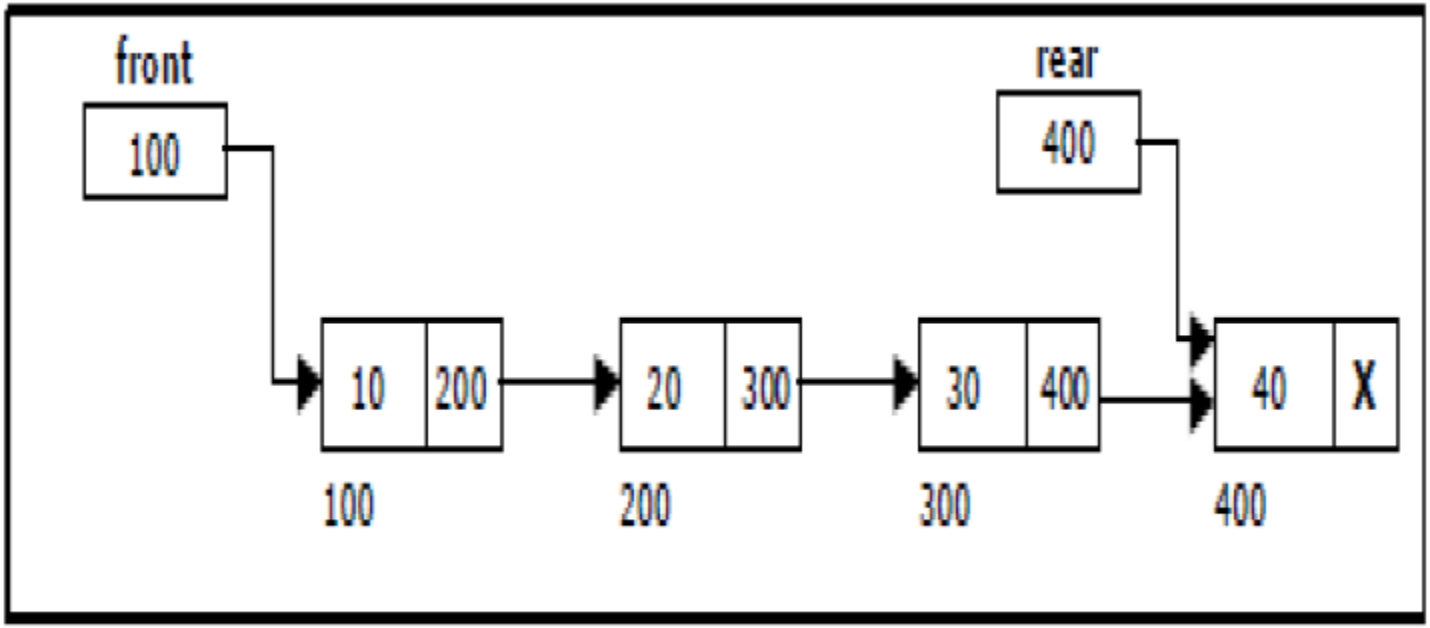


Figure : Linked Queue representation

5.3 Applications of Queue

1. It is used to schedule the jobs to be processed by the CPU.
2. When multiple users send print jobs to a printer, each printing job is kept in the printing queue. Then the printer prints those jobs according to first in first out (FIFO) basis.
3. Breadth first search uses a queue data structure to find an element from a graph.

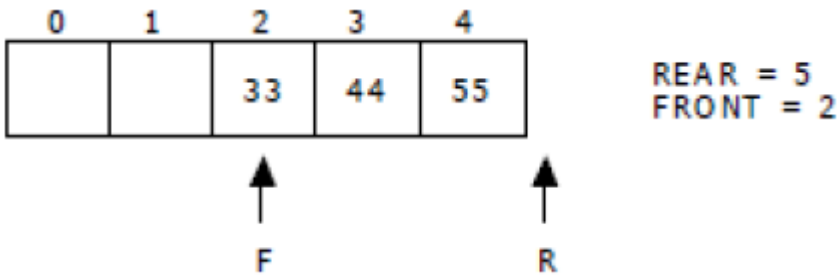
B: CIRCULAR QUEUE

- A more efficient queue representation is obtained by regarding the array Q[MAX] as circular. Any number of items could be placed on the queue. This implementation of a queue is called a circular queue because it uses its storage array as if it were a circle instead of a linear list.
- **There are two problems associated with linear queue**. They are:

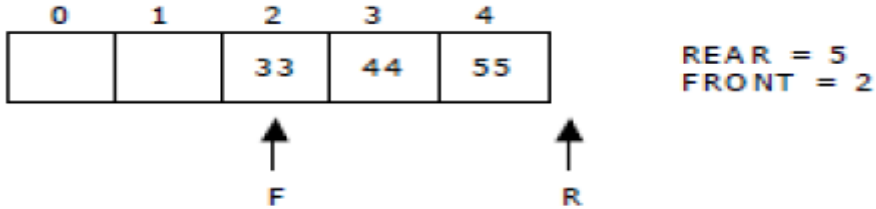
1. Time consuming: linear time to be spent in shifting the elements to the beginning of the queue.
2. Signaling queue full: even if the queue is having vacant position

Cont’

- For example, let us consider a linear queue status as follows:



- Next insert another element, say 66 to the queue. We cannot insert 66 to the queue as the rear crossed the maximum size of the queue (i.e., 5). There will be queue full signal. The queue status is as follows:

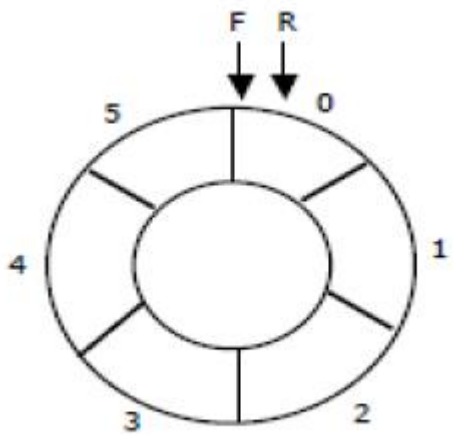


Cont’

- This difficulty can be overcome if we treat queue position with index zero as a position that comes after position with index four then we treat the queue as a **circular queue.**
- In circular queue if we reach the end for inserting elements to it, it is possible to insert new elements if the slots at the beginning of the circular queue are empty.

Representation of Circular Queue:

- Let us consider a circular queue, which can hold maximum (MAX) of six elements. Initially the queue is empty.

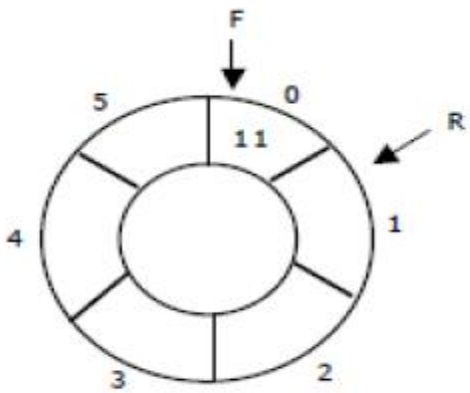


Circular Queue

Queue Empty
MAX = 6
FRONT = REAR = 0
COUNT = 0

Cont’

- Now, insert 11 to the circular queue. Then circular queue status will be:



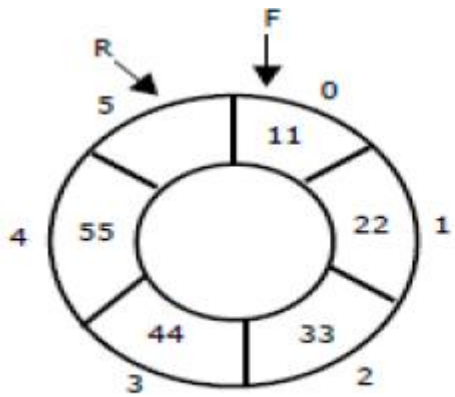
Circular Queue

FRONT = 0
REAR = (REAR + 1) % 6 = 1
COUNT = 1

- Insert new elements 22, 33, 44 and 55 into the circular queue. The circular queue status is:

Cont’

- F

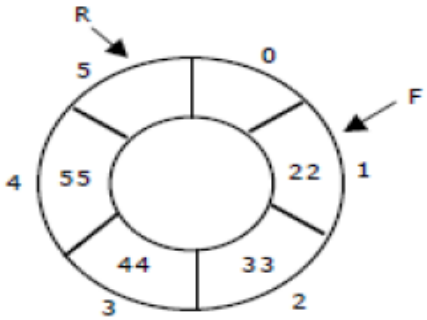


Circular Queue

FRONT = 0
REAR = (REAR + 1) % 6 = 5
COUNT = 5

- Now, delete an element. The element deleted is the element at the front of the circular queue. So, 11 is deleted. The circular queue status is as follows:

Cont’

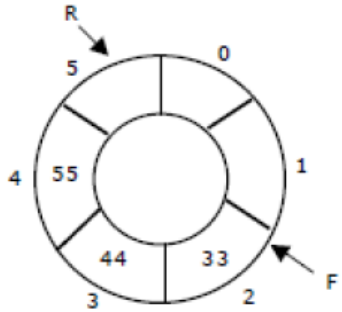


Circular Queue

FRONT = (FRONT + 1) % 6 = 1
REAR = 5
COUNT = COUNT - 1 = 4

- Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The circular queue status is as follows:

- 1



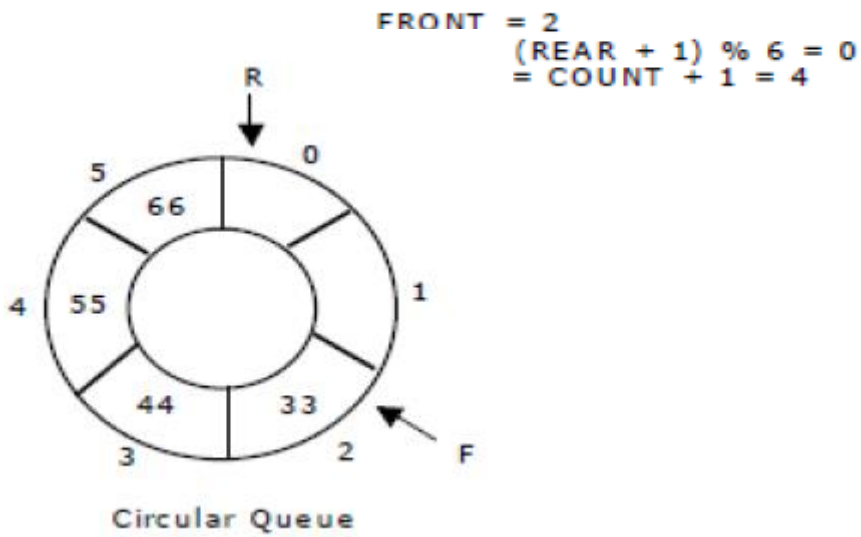
Circular Queue

FRONT = (FRONT + 1) % 6 = 2
REAR = 5
COUNT = COUNT - 1 = 3

Cont’

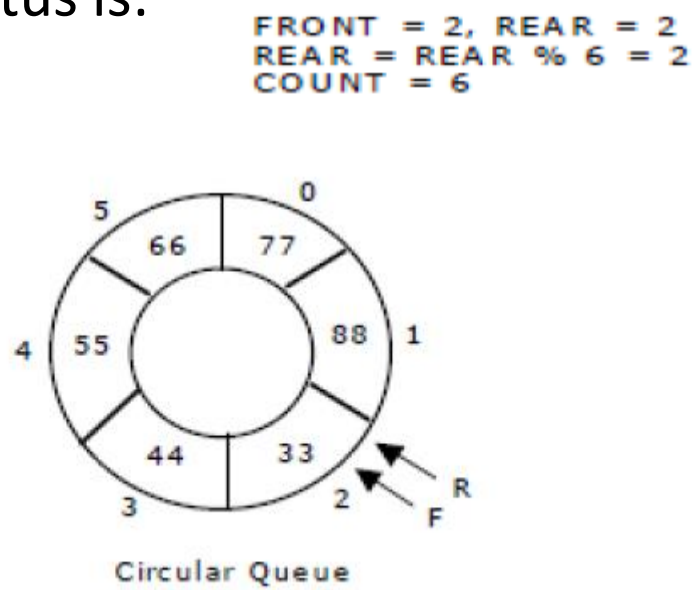
next

- Again, insert another element 66 to the circular queue. The status of the circular queue is:



next

- Now, insert new elements 77 and 88 into the circular queue. The circular queue status is:



Deque

- In this section we examine an extension of the queue, which provides a means to insert and remove items at both ends of the queue. This data structure is a ***de-queue***. The word ***de-queue*** is an acronym derived from ***double-ended queue***. Below figure shows the representation of a dequeue.

Cont’

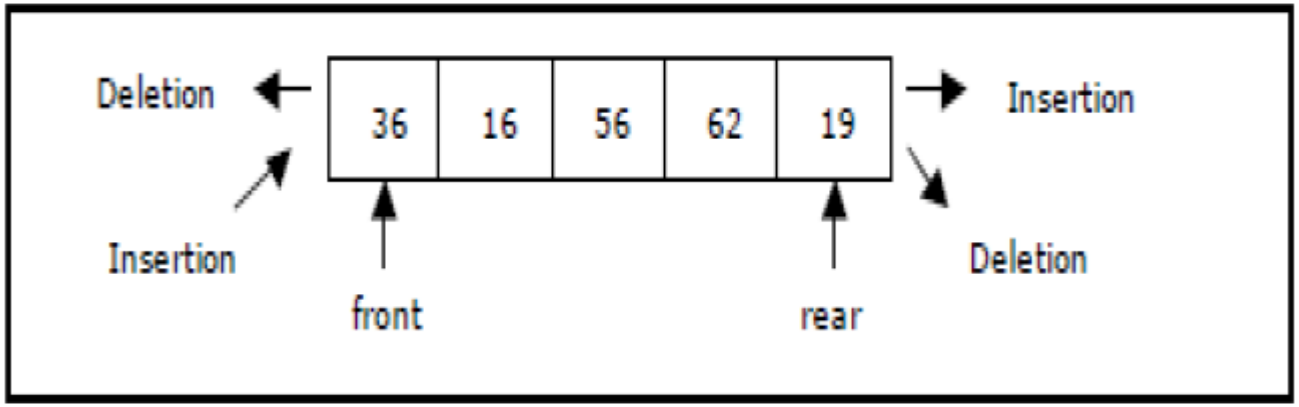


Figure Representation of a deque.

- deque provides four operations. Below Figure shows the basic operations on a deque.
- • enqueue_front: insert an element at front.
- • dequeue_front: delete an element at front.
- • enqueue_rear: insert element at rear.
- • dequeue_rear: delete element at rear.

Cont’

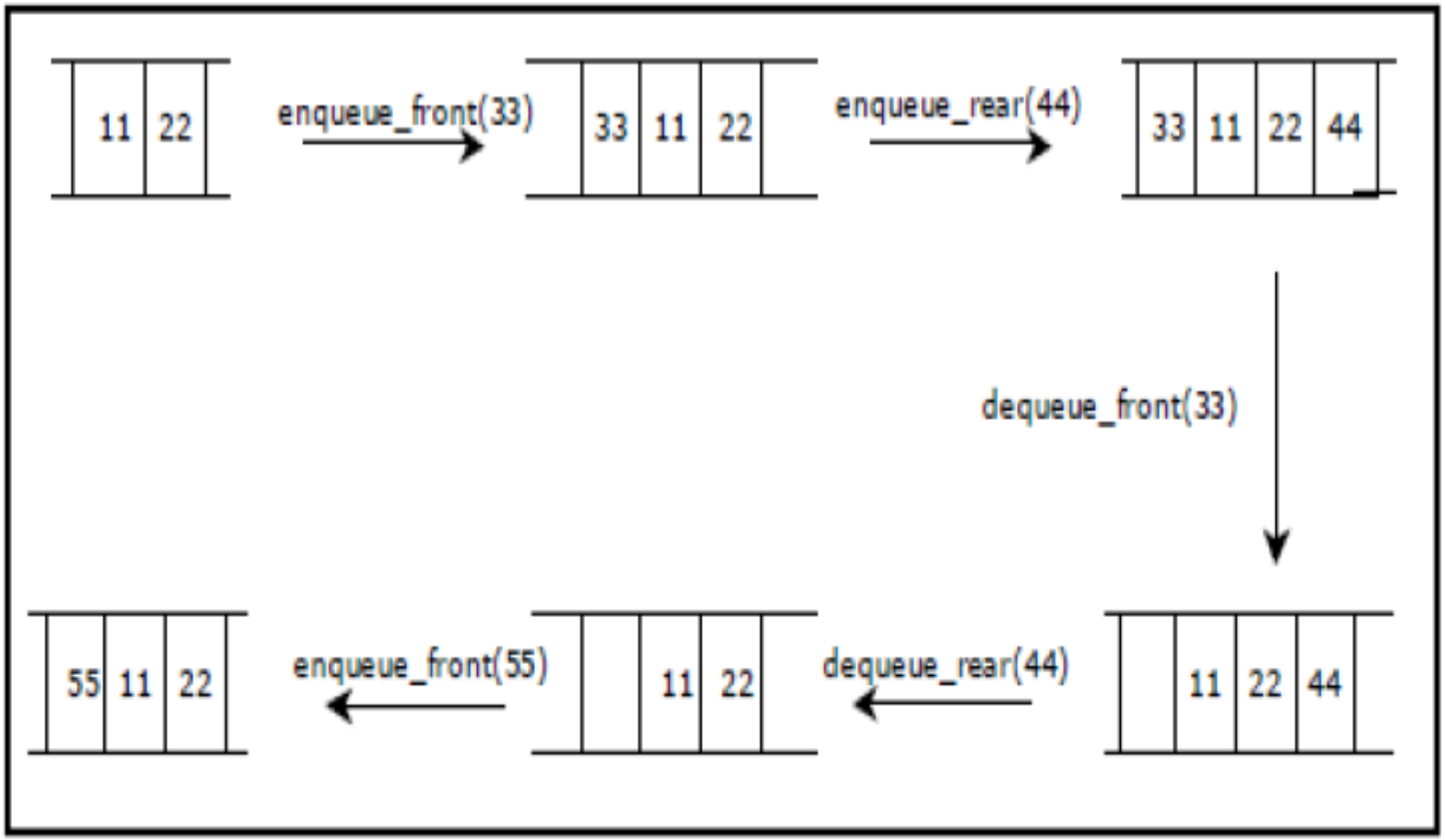
- 

Figure ˙ ˜. Basic operations on deque

Cont'

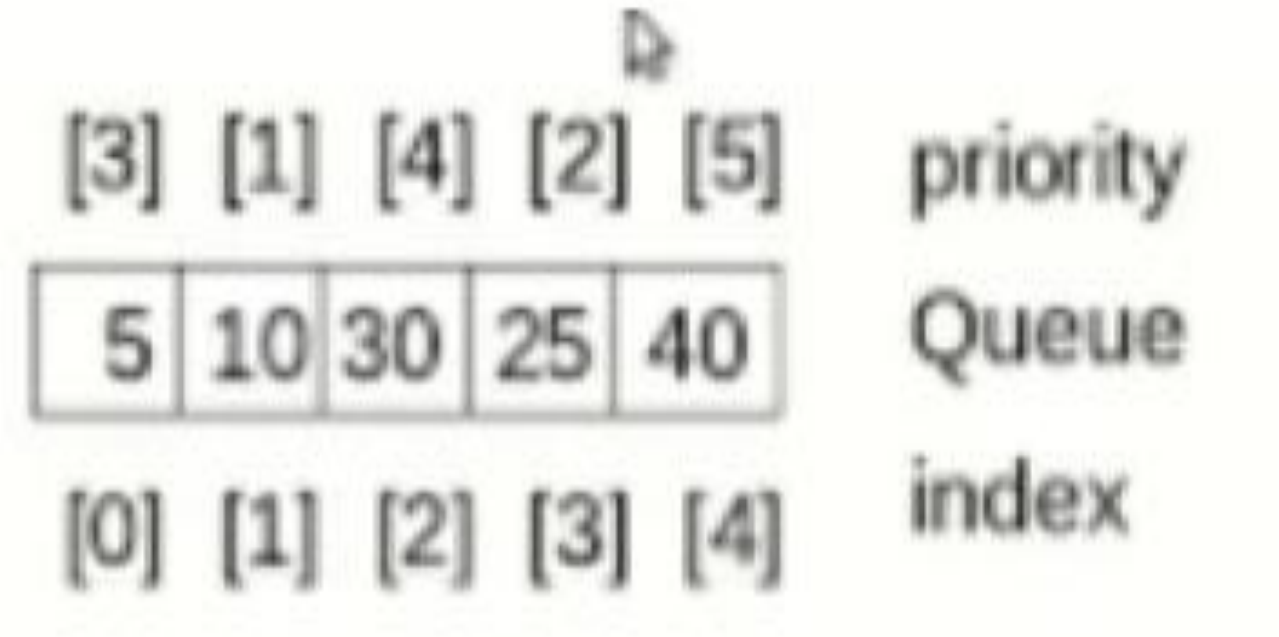
- There are two variations of de-queue. They are:
  - Input restricted de-queue (IRD)
  - Output restricted de-queue (ORD)
- An Input restricted de-queue is a de-queue, which allows insertions **at one end** but allows deletions at both ends of the list.
- An output restricted de-queue is a de-queue, which allows deletions at one end but allows insertions at both ends of the list.
- **Priority Queue:**

Cont’

- A **priority queue** is a collection of elements such that each element has been assigned a priority. We can insert an element in priority queue at the rare position. We can delete an element from the priority queue based on the elements priority and such that the order in which elements are deleted and processed comes from the following rules:
- 1. An element of **higher priority** is processed before any element of lower priority. [Example of bank customer who are served based on the amount of their deposit, think on this system]
- 2. Two elements with same priority are processed according to the order in which they were added to the queue. It follows FIFO or FCFS(First Comes First serve) rules.

Cont’

- We always remove an element with the highest priority, which is given by the minimal integer priority assigned.



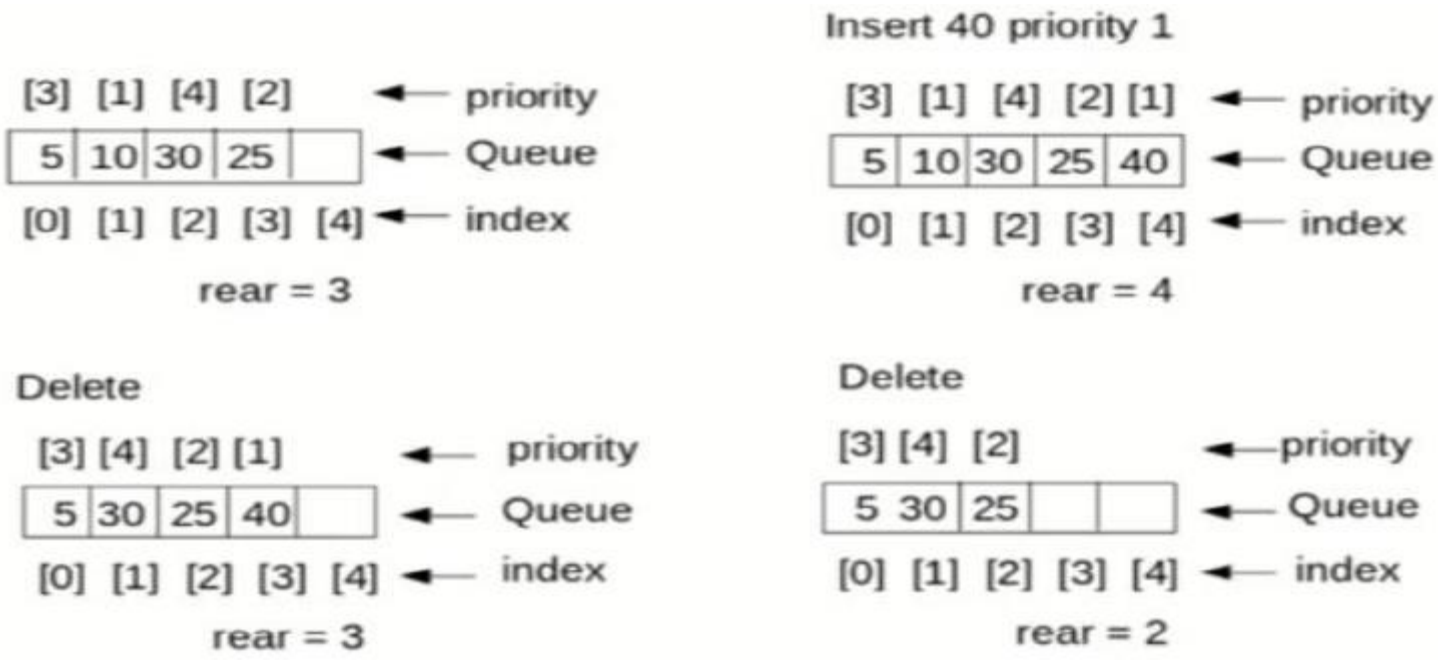
Cont’

- A prototype of a priority queue is **time sharing system**: programs of high priority are processed first, and programs with the same priority form a standard queue. An efficient implementation for the Priority Queue is to use heap, which in turn can be used for sorting purpose called heap sort
- **Priority queues are two types:**
- 1. Ascending order priority queue
- 2. Descending order priority queue

- **1. Ascending order priority queue:** It is **Lower priority number to high priority number.**
- Examples: order is 1,2,3,4,5,6,7,8,9,10
- **2. Descending order priority queue:** It is **high priority number to lowest priority number.**
- Examples: Order is 10,9,8,7,6,5,4,3,2,1

1.Through Queue (Using Array):

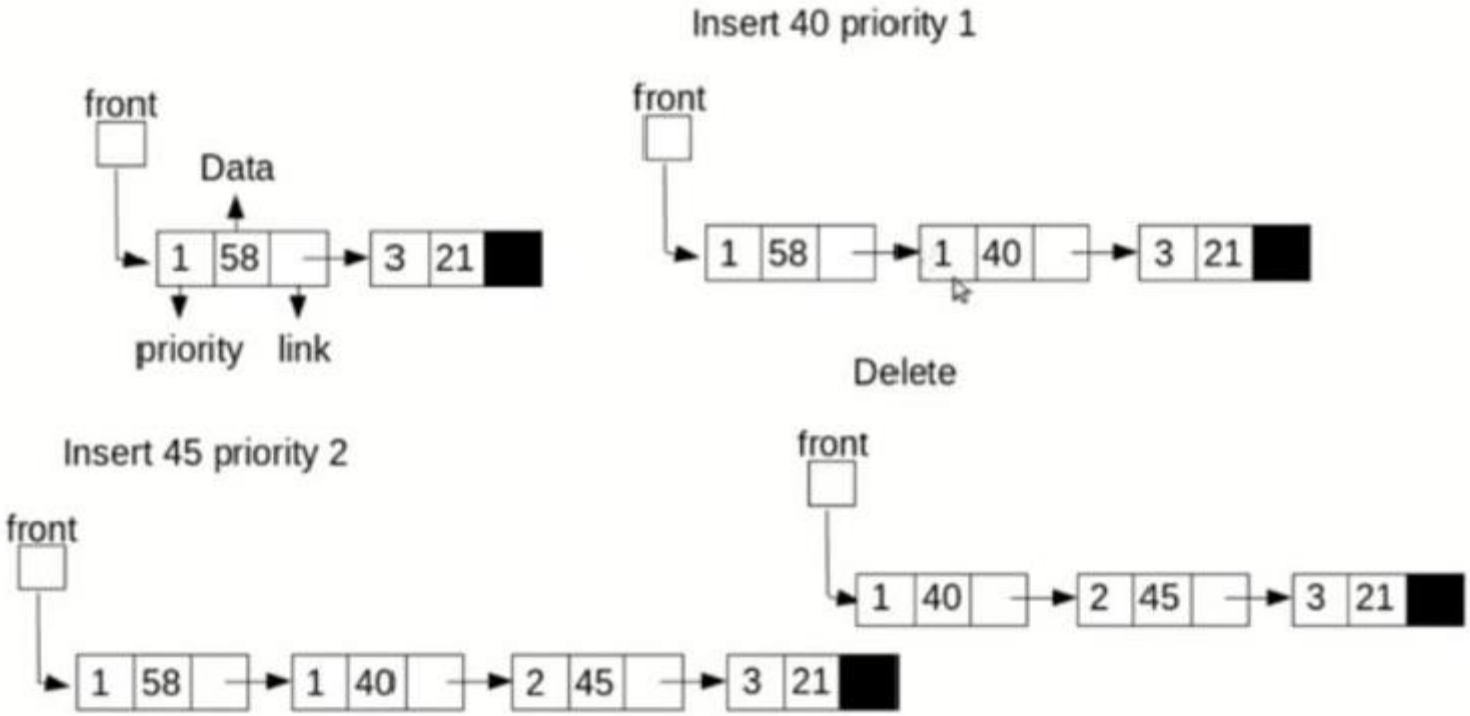
- In this case element is simply added at the rear end as usual. For deletion, the element with highest priority is searched and then deleted.



2. Through sorted List (Using Linked List):

- In this case insertion is costly because the element insert at the proper place in the list based on the priority. Here deletion is easy since the element with highest priority will always be in the beginning of the list.

Cont’

- 
front
Data
1
58
3
21
priority
link
Insert 40 priority 1
front
1
58
1
40
3
21
Delete
front
1
40
2
45
3
21
Insert 45 priority 2
front
1
58
1
40
2
45
3
21

Difference between stacks and Queues

Stack

1.A stack is a linear list of elements in which the element may be inserted or deleted at one end.

2. In stacks, elements which are inserted last is the first element to be deleted.

3.Stacks are called LIFO (Last In First Out)list

4.In stack elements are removed in reverse order in which thy are inserted.

5.suppose the elements a,b,c,d,e are inserted in the stack, the deletion of elements will be e,d,c,b,a.

Queue

1.A Queue is a linear list of elements in which the elements are added at one end and deletes the elements at another end.

2. . In Queue the element which is inserted first is the element deleted first.

3. Queues are called FIFO (First In First Out)list.

4. In Queue elements are removed in the same order in which thy are inserted.

5. Suppose the elements a,b,c,d,e are inserted in the Queue, the deletion of elements will be in the same order in which thy are inserted.

Cont’

Stack

6.In stack there is only one pointer to insert and delete called “Top”.

7.Initially top=-1 indicates a stack is empty.

8.Stack is full represented by the condition TOP=MAX-1(if array index starts from ‘0’).

9.To push an element into a stack, Top is incremented by one

10.To POP an element from stack, top is decremented by one.

Queue

6. In Queue there are two pointers one called “Rear” and another for deletion for insertion called “Front”.

7. Initially Rear=Front=-1 indicates a Queue is empty.

8.Queue is full represented by the condition Rear=Max-1.

9.To insert an element into Queue, Rear is incremented by one.

- 10.To delete an element from Queue, Front is

GROUP ASSIGNMENT

Design 5 web pages that contain the following information: [Application of linear DS]

1. 1st page includes head "**Our registration numbers** > put some information about your campus
2. Create a database in MySQL, SQLite or oracle, and then add fields names :First name, last name , creation date, and your photo. Connect your database to HTML template that has the button to control your photos, **next, back , last ,first,** to access them
3. On the same template add "**REDO**" function to bring back the a deleted photo from your database
4. 4th page includes "sub header, **STACK ,Queue**"

On the same page Create a stack in and Queue that allow user to push or pop the data.

5. link all pages by using both double and circular links